人工智能与深度学习

机器学习（二）

华中科技大学 机械科学与工程学院
陈伟

2026

二元分类的度量指标

Measures for binary classification

二元分类问题：示例

- **二元：True/False**

- **病例诊断问题**
  - 诊断病例是否得了某种疾病
  - “二元”：阳性 vs. 阴性

- **（简化的）信息检索问题**
  - 我们有很多条信息，要从中选出用户感兴趣的信息
  - “二元”：用户感兴趣的 vs. 用户不感兴趣的

示例

- 单看标题，我们能了解到什么



示例

- **“准确率=90%”是什么意思?**
- **我们假定**

$$\text{准确率} = \frac{\text{正确的诊断数}}{\text{总诊断数}}$$

- **“准确率=90%”一定很厉害吗?**

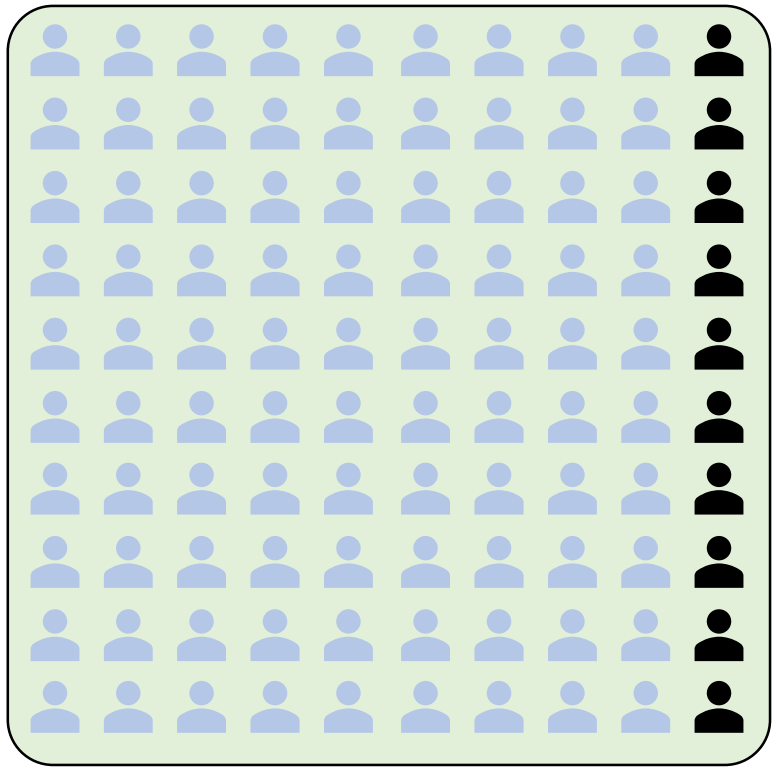
示例

- 假如，所有输入到模型诊断的样本中，有10%实际为阳性，有90%实际为阴性

- 如何才能达到“准确率=90%”？

示例

- 如果一个模型对无论什么输入，都输出为阴性，它的“准确率”就是90%！





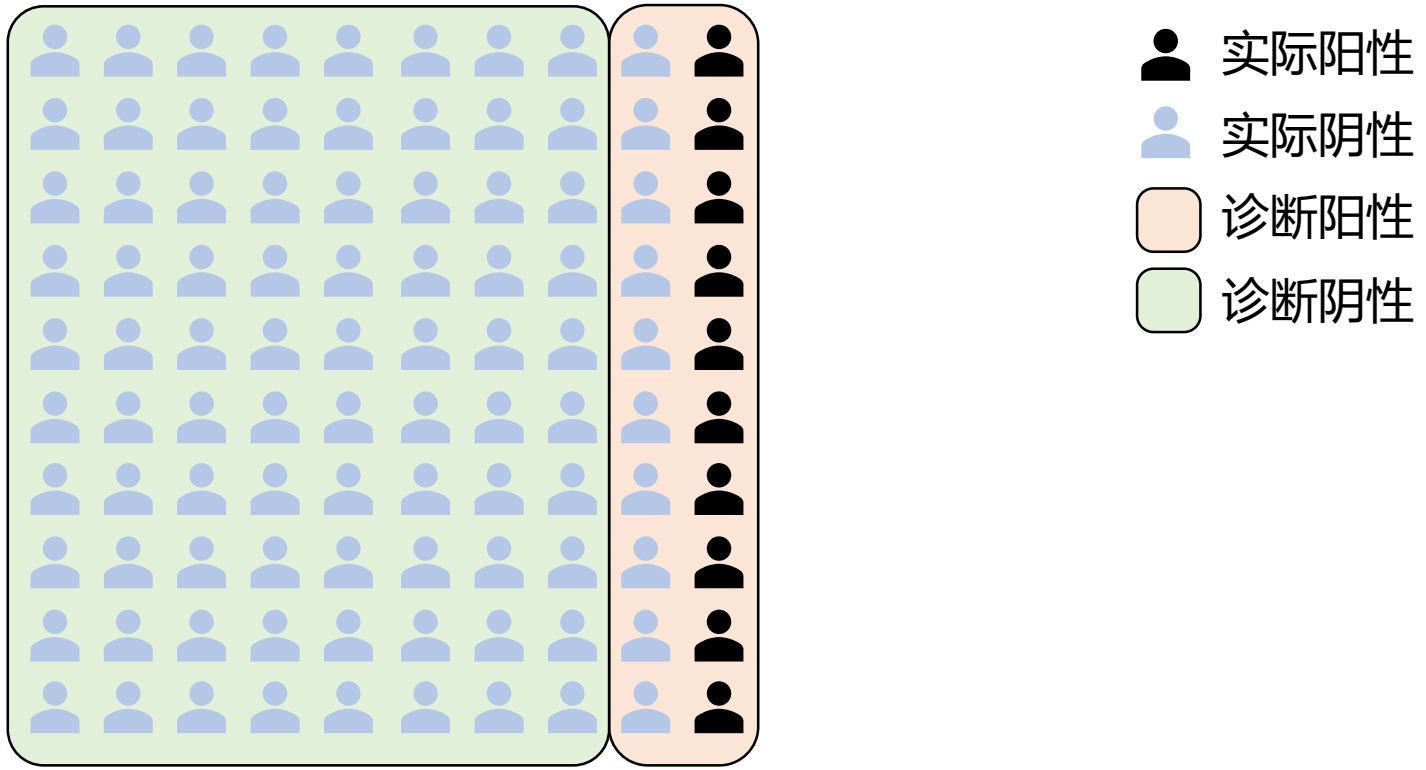
测试数据的类型分布很重要

- **对二元分类问题，我们需要知道测试数据中各类型的比例，才能评价测试结果**

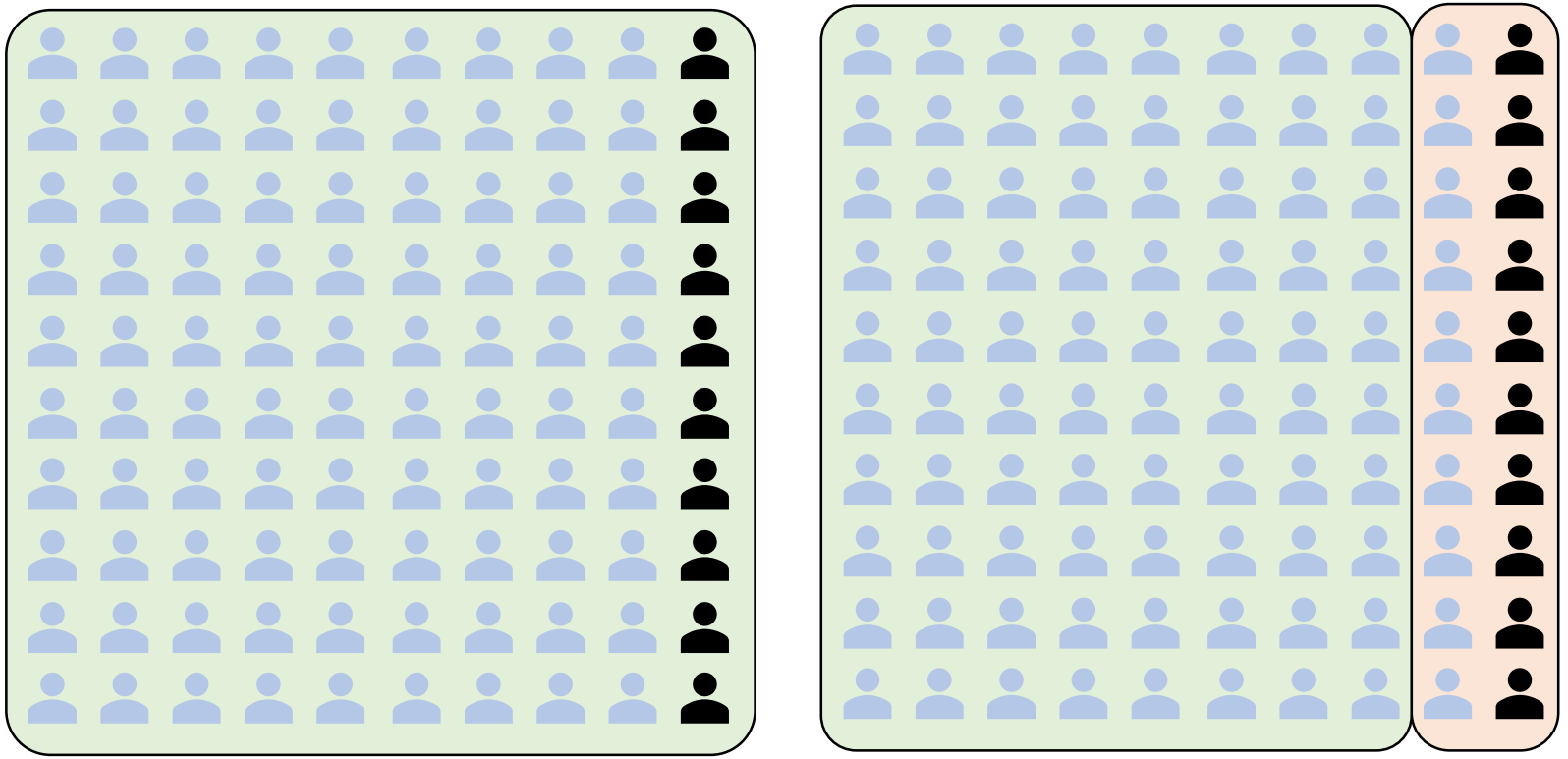
- **但是，“准确率+比例”是否就足以评价结果呢?**
  - 换句话说，假如已知测试数据中的实际阳性率=10%，模型在测试数据上的准确率为90%，这个模型是不是一定很糟糕?

示例

- **假设有个模型能准确地捕捉到所有阳性样本，但也有1/9的概率把真阴性误判称阳性（假阳性）**
  - 此模型比上一个模型有用多了，尽管它的“准确率”依旧只有90%



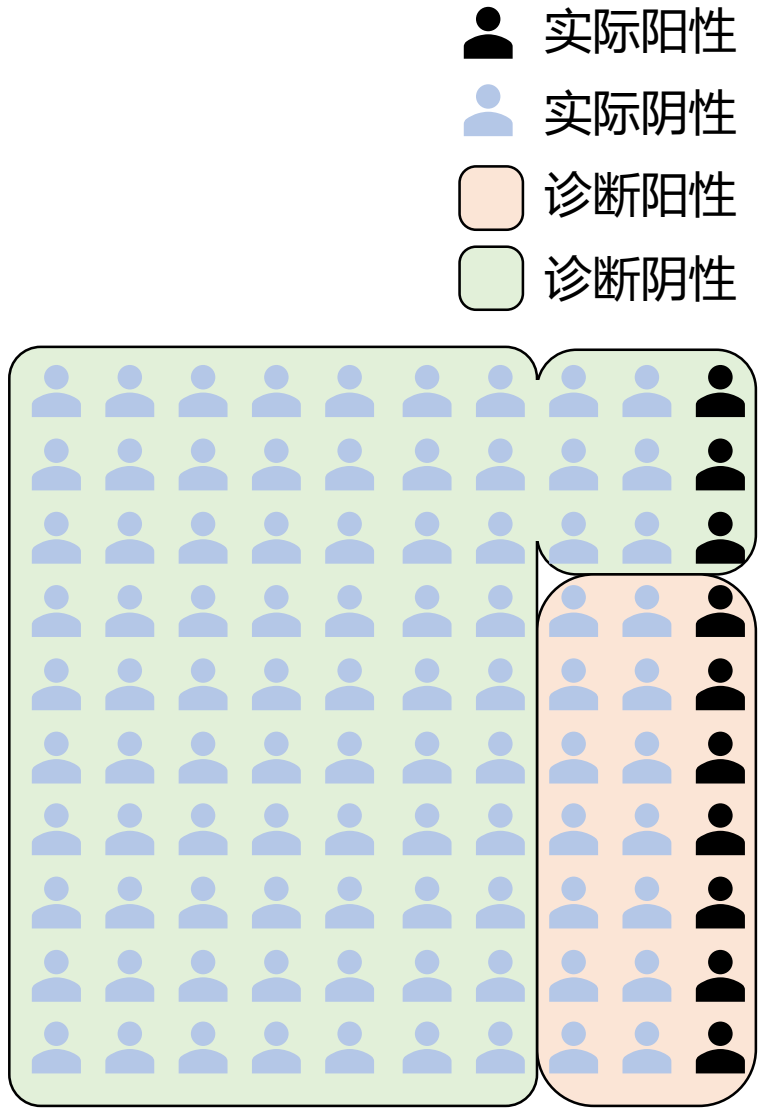
如何全面评价二元分类模型



- **在评价二元模型时，仅仅用“准确率+比例”是不够的**
  - 上图两种情况都是在10%阳性率测试集上达到90%准确率
- **我们还要引入新的数据**

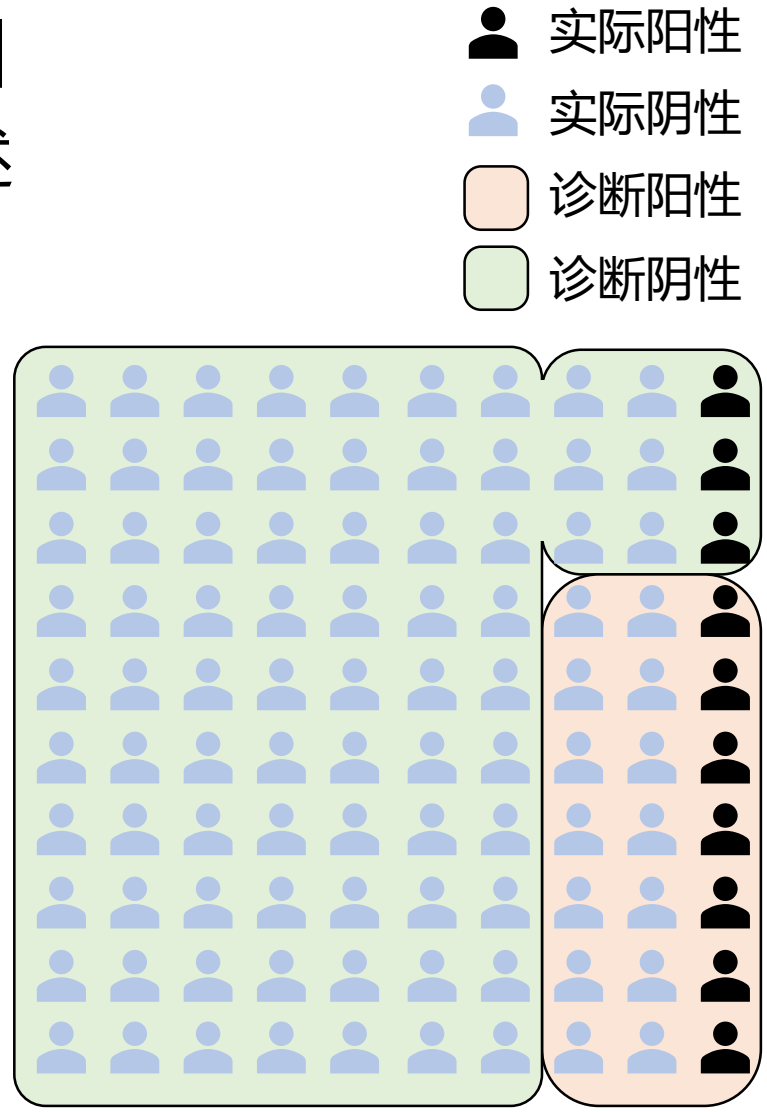
全面评价二元分类模型

- **如何全面描述右图?**
  - 图可以分为四个部分
    - 实际阳性且诊断阳性：7
    - 实际阳性且诊断阴性：3
    - 实际阴性且诊断阳性：14
    - 实际阴性且诊断阴性：76
  - 显然，上述4个数字可全面描述这幅图
  - 事实上，必须要4个数字才能全面描述这幅图



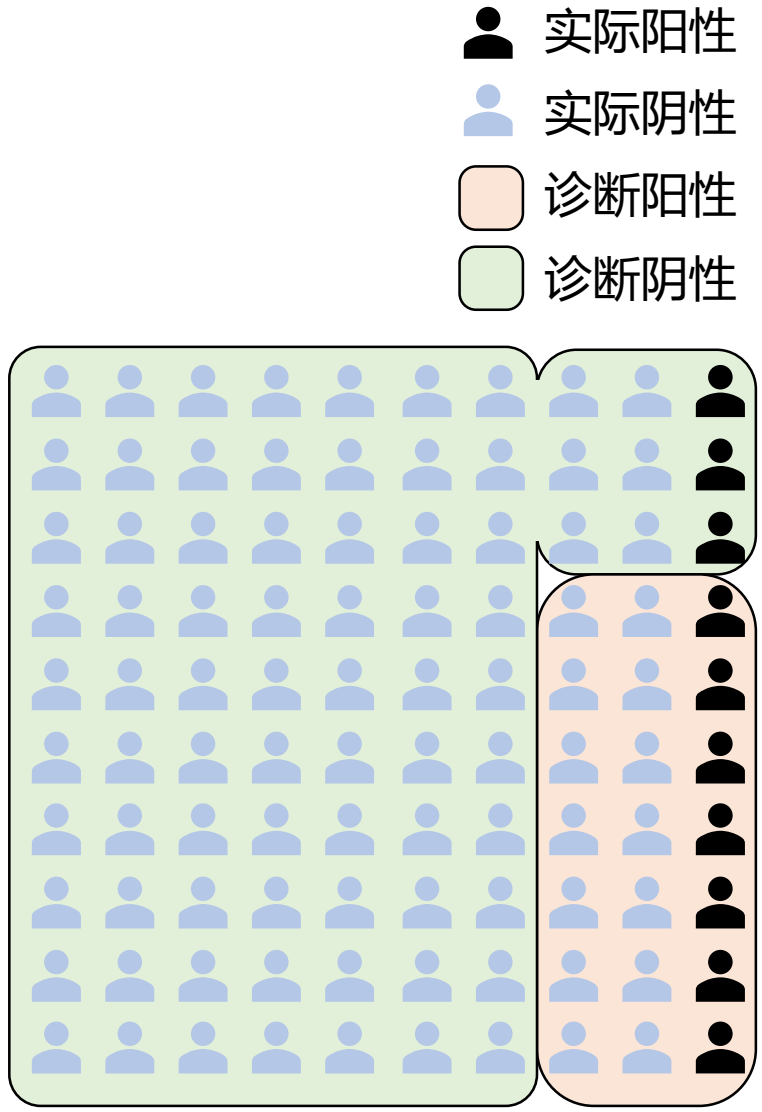
全面评价二元分类模型

- **4个数字才能全面描述右图**
  - 我们也可选择另外一组描述
    - 实际阳性且诊断阳性：7
    - 实际阳性且诊断阴性：3
    - 实际阴性且诊断阳性：14
    - 总样本数：100
  - 或者
    - 总样本数：100
    - 实际阳性：10
    - 诊断阳性：21
    - 实际阳性且诊断阳性：7



全面评价二元分类模型

- **4个数字才能全面描述右图**
  - 3个百分比也可以
    - 实际阳性且诊断阳性：7%
    - 实际阳性且诊断阴性：3%
    - 实际阴性且诊断阳性：14%
  - 或者
    - “准确率”：83%
      - 7% + 76%
    - 实际阳性且诊断阳性：7%
    - 实际阳性且诊断阴性：3%



评价二元分类模型

- **有很多性能度量指标可以用来评价二元分类模型**
  - 稍后我们会介绍其中常用的一些

- **至少要4个数字指标，或者3个百分比指标，才能复现“实际/诊断-阳性/阴性”组合图，才能全面评价二元分类的效果**

度量指标：精度（“准确率”）与错误率

- **精度是分类正确的样本数占样本总数的比例**

$$\text{精度（准确率）} = \frac{\text{分类正确的样本数}}{\text{总样本数}}$$

- **错误率是分类错误的样本数占样本总数的比例**

$$\text{错误率} = \frac{\text{分类错误的样本数}}{\text{总样本数}}$$

$$\text{错误率} = 1 - \text{精度}$$

度量指标：分类结果混淆矩阵（confusion matrix）

- **根据样本真实类别与学习预测类别的组合，可以得到以下四种类型的样本**
  - 真正例（true positive，TP）
  - 假正例（false positive，FP）
  - 真负例（true negative，TN）
  - 假负例（false negative，FN）

T（真）代表模型预测对了
F（假）代表模型预测错了

P（正）代表模型输出为正
N（负）代表模型输出为负

例如，检测中的“假阳性”就是FP

真实情况	预测情况	
	正例	负例
正例	TP（真正例）	FN（假负例）
负例	FP（假正例）	TN（真负例）

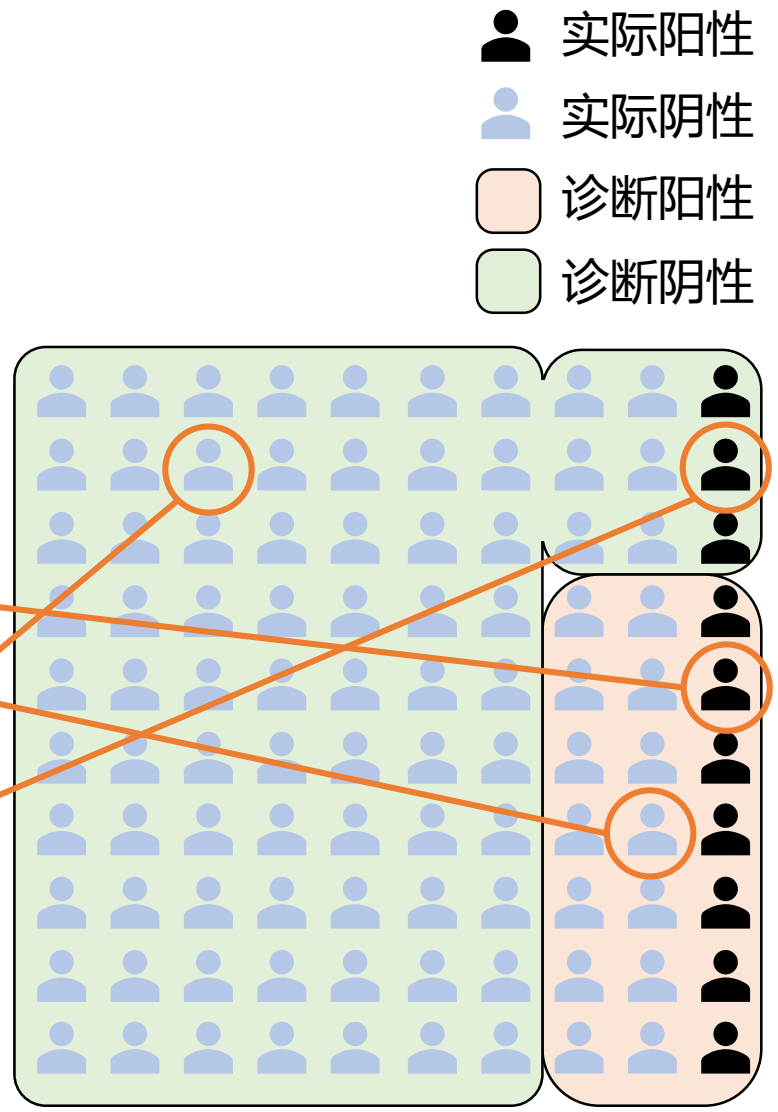
度量指标：分类结果混淆矩阵（confusion matrix）

T（真）代表模型预测对了
F（假）代表模型预测错了

P（正）代表模型输出为正
N（负）代表模型输出为负

例如，检测中的“假阳性”就是FP

- 真正例（true positive，TP）
- 假正例（false positive，FP）
- 真负例（true negative，TN）
- 假负例（false negative，FN）



度量指标：查准率与查全率

- **查准率 P（precision）：所有预测阳性中，真实阳性的比例**

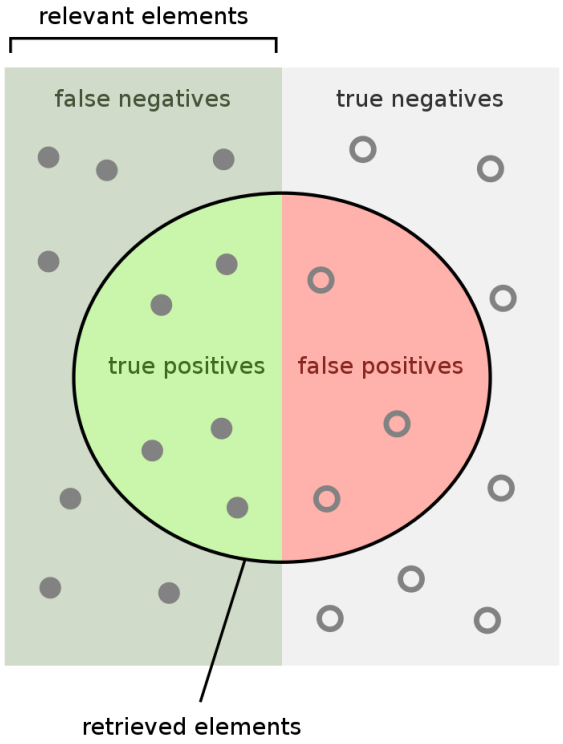
$$P = \frac{TP}{TP + FP}$$

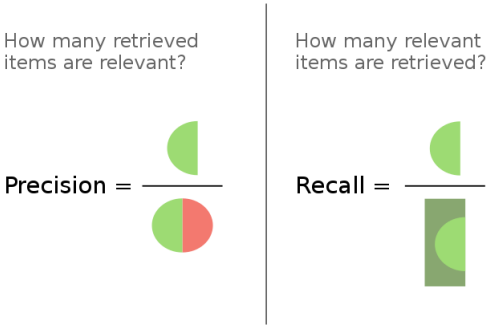
  - “检索出的信息中，有多少比例是用户感兴趣的”

- **查全率 R（recall，sensitivity）：所有真实阳性中，预测阳性的比例**

$$R = \frac{TP}{TP + FN}$$

  - “用户感兴趣的信息中，有多少被检索出来了”





度量指标：查准率与查全率

- **在选择分类模型或者调整分类模型参数时，查准率和查全率往往是一对矛盾的变量**
  - 一个模型的查准率高，往往查全率偏低；反之亦然
  - 在检索问题中，可通过增加提供的信息数量（模型多预测一些“+值”）来实现增加查全率；但把所有信息都提供出来，会降低查准率
  - 我们也可以只提供“最有把握”的信息，来提高查准率；但这会降低查全率

度量指标与模型选择

- **在很多情形下，我们可根据模型的预测结果对样例进行排序**
  - 排在前面的是模型认为“最可能”是正例的样本
  - 排在后面的是模型认为“最不可能”是正例的样本

- **分类过程就相当于在这个排序中以某个“截断点”将样本分为两类；我们可以通过调节截断点的位置（将其看作模型参数）来调节“输出正例”的标准**

度量指标与模型选择

- **对模型选择的几种理解：**

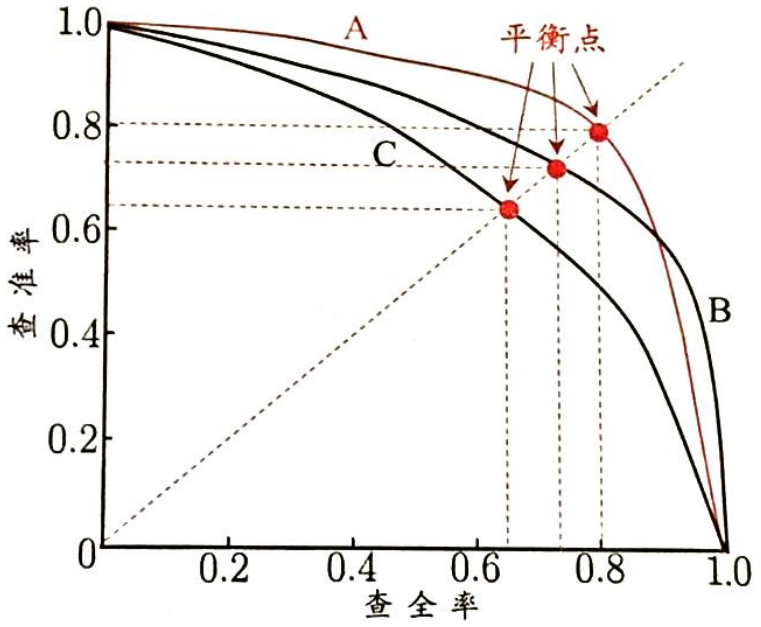
  - 一个模型的输出不是二元的0/1，而是一个[0,1]之间的值，我们需要人工选择一个“截断点”：该点以上认为是1，以下认为是0；我们想评价这个模型（将其与其它模型比较）

  - 我们有一组模型（比如采用不同L1正则化参数的线性分类器），输出都是二元的0/1，它们的效果各不相同，我们想评价这一系列模型（将其与其它组模型比较）

模型的度量指标：P-R曲线与平衡点

- **对于不同的“输出正例标准”，我们可以得到不同的查准率和查全率**
  - 以查准率为纵轴、查全率为横轴作图，就得到了P-R曲线

- **平衡点（Break-even point）是P-R曲线上“查准率=查全率”时的取值**
  - 平衡点高的模型，通常优于平衡点低的模型

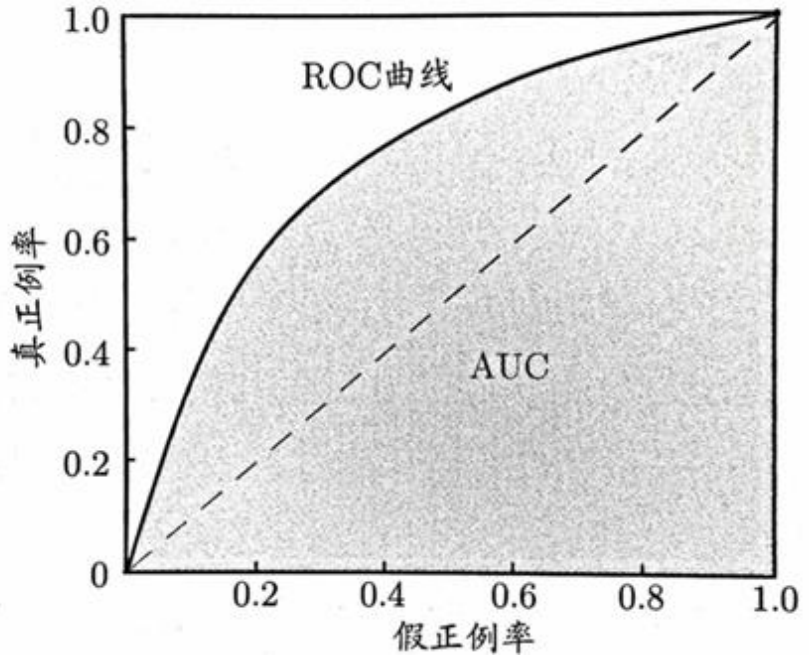


模型的度量指标：ROC和AUC

- **对于不同的“输出正例标准”，我们也可以绘制其它形式的曲线**
  - 横轴使用“假正例率”（false positive rate，FPR），纵轴使用“真正例率”（true positive rate，TPR），我们画出的曲线叫“ROC曲线”
    - ROC曲线中的TPR就是查全率，但FPR是一个新定义的指标

$$FPR = \frac{FP}{TN + FP}$$

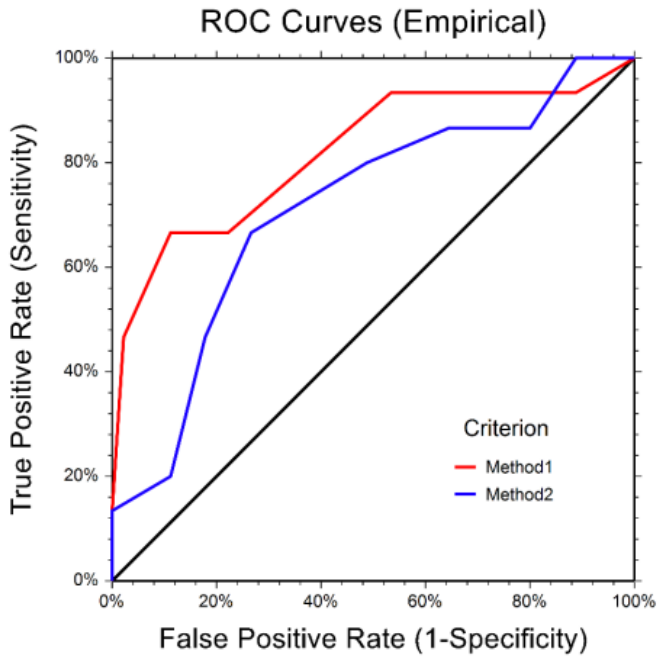
$$TPR = \frac{TP}{TP + FN}$$



模型的度量指标：ROC和AUC

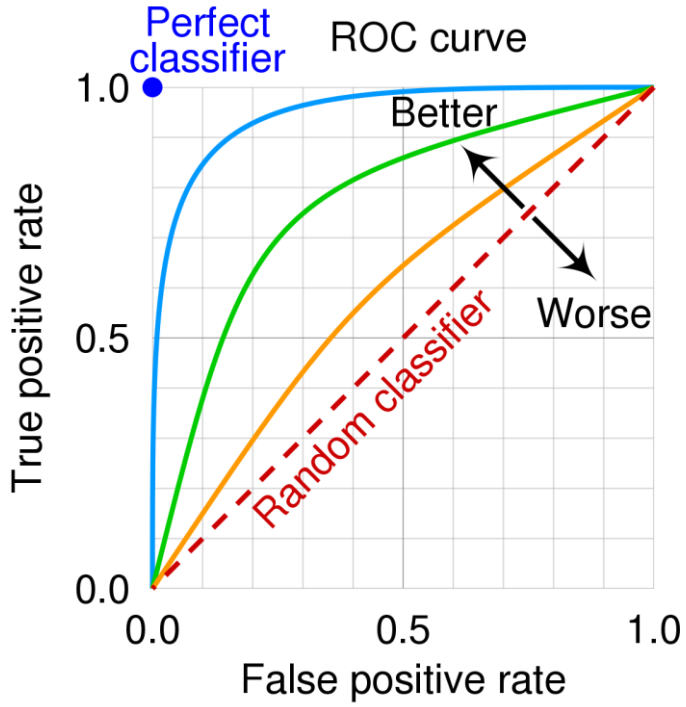
- ROC全称是“受试者工作特征”（Receiver Operating Characteristics），源于二战中对敌机检测的雷达信号分析技术

  - 实际通过数据计算得到的通常是ROC曲线上的一些点；连接这些点，得到ROC曲线



模型的度量指标：ROC和AUC

- **若一个机器学习模型的曲线完全在另一个模型的左上方，则前者更优**
  - 完美的模型位于（0，1）



- **若两条线有交叉，则一般难以断言哪个模型更好**
  - 一个可以使用的判据，是ROC曲线下的面积，即AUC（area under ROC curve）

$$FPR = \frac{FP}{TN + FP}$$

$$TPR = \frac{TP}{TP + FN}$$

度量指标：F1度量

- **如果一定要用一个单一指标来评价、比较二元分类效果，较常用的指标是F1度量（F1 score）**

$$F1 = \frac{2 \times P \times R}{P + R} = \frac{2 \times TP}{\text{总样本数} + TP - TN}$$

  - F1 是基于查准率与查全率的调和平均

$$\frac{1}{F1} = \frac{1}{2}\left(\frac{1}{P} + \frac{1}{R}\right)$$

  - 相对于算术平均和几何平均，调和平均更加重视较小值

度量指标：F_β度量

- **有时我们对查准率和查全率的重视不同，这时可使用F1度量的一般形式** F_β
  - 推荐商品时，想尽可能少打扰用户：查准率更重要
  - 抓逃犯时，想尽可能避免漏掉：查全率更重要

$$F_\beta = \frac{(1+\beta^2) \times P \times R}{(\beta^2 \times P) + R}$$

$\beta > 1$时查全率有更大影响，$\beta < 1$时查准率有更大影响

- F_β是基于查准率与查全率的加权调和平均

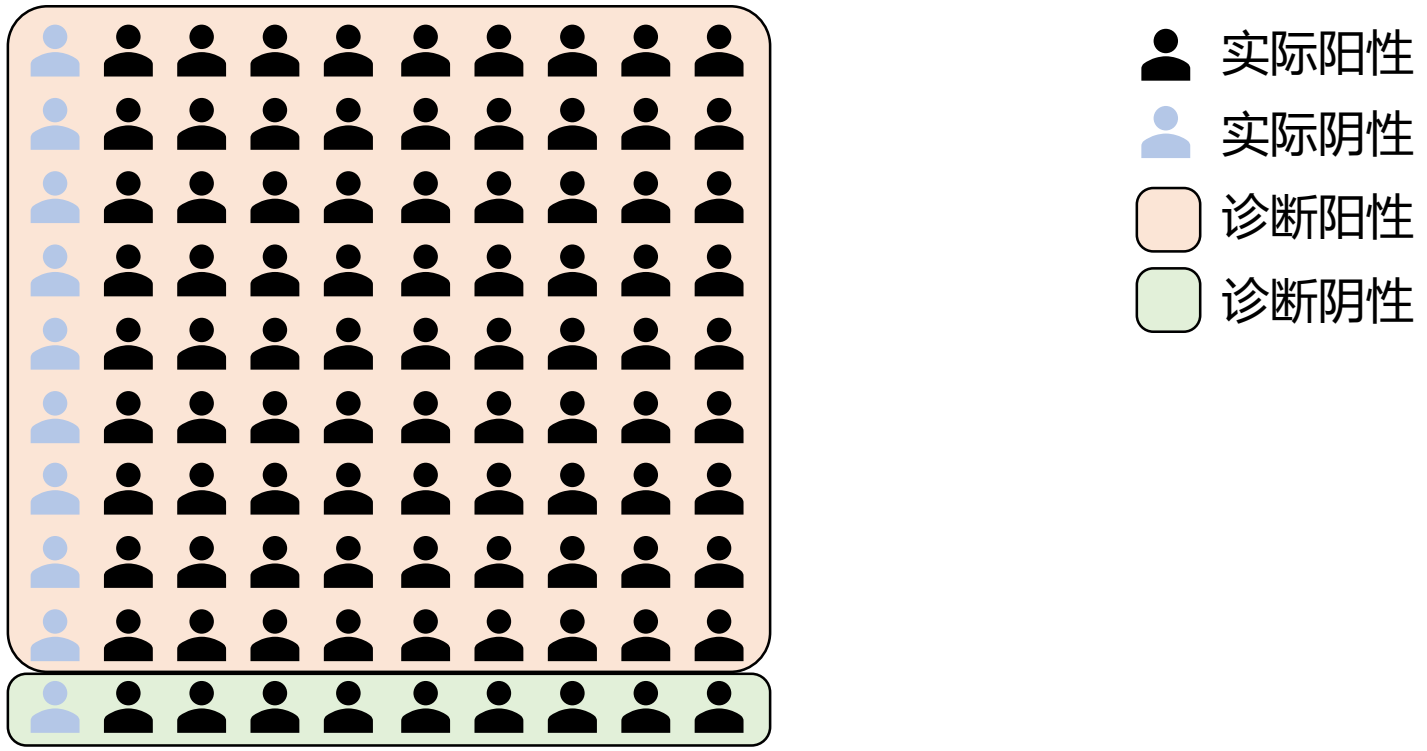
$$\frac{1}{F_\beta} = \frac{1}{1+\beta^2}\left(\frac{1}{P} + \frac{\beta}{R}\right)$$

度量指标

- 提问：某故障诊断模型，查准率=90%，查全率=90%，模型就一定很好吗?
- 记住：4个数字，或3个百分比!
- 如果可以让我挑选测试数据集，我可以轻松达到“查准率=90%，查全率=90%”（无论什么二元分类任务）

度量指标

- **查准率=90%，查全率=90%**



- 如何“轻松”得到这样的模型：不使用任何特征，对每个样本，90%概率输出正，10%概率输出负（“盲猜”模型）

度量指标

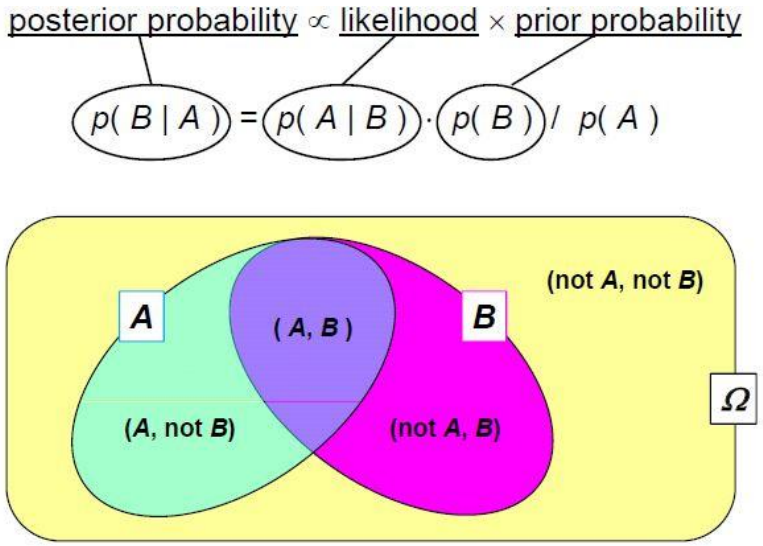
- **记住：4个数字，或3个百分比！**

- **有时问题会包含默认的、隐藏的数据信息**
- **如“0~9手写数字识别，准确率98%”**（注：这是个多元分类问题，不是二元分类问题）
  - 默认信息：测试数据集上0~9每个数字出现概率大约均为10%
  - 一般场合，我们可以接受这些“默认信息”；而在需要十分严谨的场合，这些“默认的”信息仍然需要确认后才能认为有效

二元分类与贝叶斯定理

- **贝叶斯定理**

$$\mathbb{P}(B|A) = \frac{\mathbb{P}(B \cap A)}{\mathbb{P}(A)}$$
$$= \frac{\mathbb{P}(A|B)\mathbb{P}(B)}{\mathbb{P}(A)}$$



- **贝叶斯定理可以与二元分类预测问题联系起来**
  - 事件A：真值为正
  - 事件B：预测为正

二元分类与贝叶斯定理

- **考虑一种检查某癌症的方法**
  - 若得了这种癌症，检查结果呈阳性的概率为90%
  - 若没有得这种癌症，检查结果呈阴性的概率为90%（换句话说，若没得癌症，检查结果假阳性的概率为10%）

- **一个人在没有任何症状的情况下，到医院检查，结果呈阳性；那么，这个人患癌症的概率是多少?**

二元分类与贝叶斯定理

- **考虑一种检查某癌症的方法**
  - 若得了这种癌症，检查结果呈阳性的概率为90%
  - 若没有得这种癌症，检查结果呈阴性的概率为90%（换句话说，检查结果假阳性的概率为10%）

$$\frac{TP}{TP+FN}=0.9 \qquad \frac{TN}{TN+FP}=0.9$$

- **这个人患癌症的概率是后验概率**

$$\frac{TP}{TP+FP}=?$$

二元分类与贝叶斯定理

- 只给了2个百分比，缺条件

- 补充条件：假设该癌症的发病率是1%

$$\frac{TP+FN}{TP+FP+TN+FN}=0.01$$

- 令 $TP+FP+TN+FN=100\%$

二元分类与贝叶斯定理

- $$\frac{TP}{TP+FN}=0.9 \qquad \frac{TN}{TN+FP}=0.9$$

$$\frac{TP+FN}{TP+FP+TN+FN}=0.01$$

$$TP+FP+TN+FN=100\%$$

- **线性方程**

$$\begin{bmatrix} 0.1 & 0 & 0 & -0.9 \\ 0 & -0.9 & 0.1 & 0 \\ 0.99 & -0.01 & -0.01 & 0.99 \\ 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} TP \\ FP \\ TN \\ FN \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

二元分类与贝叶斯定理

- 解线性方程得到

$$\begin{bmatrix} TP \\ FP \\ TN \\ FN \end{bmatrix} = \begin{bmatrix} 0.009 \\ 0.099 \\ 0.891 \\ 0.001 \end{bmatrix}$$

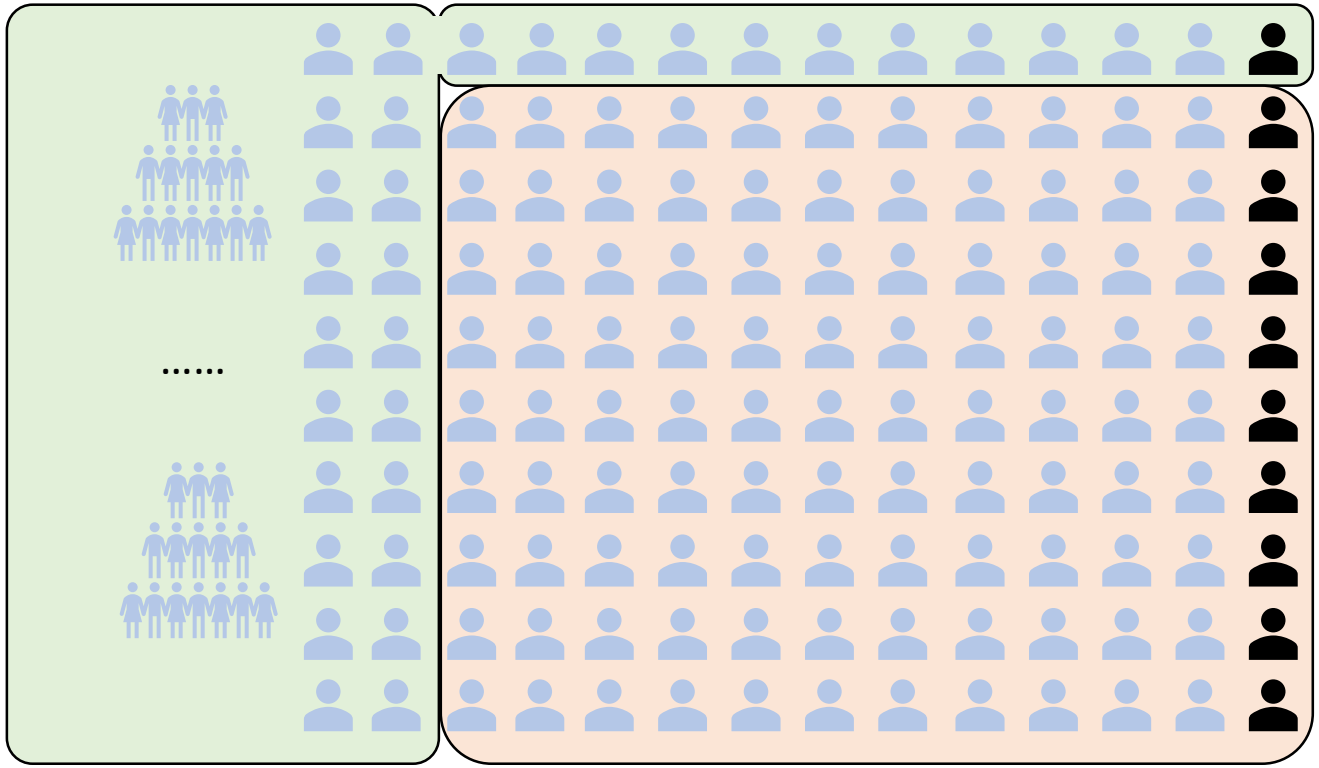
- 一个人在没有任何症状的情况下，到医院检查，结果呈阳性；这个人患癌症的概率是

$$\frac{TP}{TP + FP} = 8.33\%$$

二元分类与贝叶斯定理

- 若得了这种癌症，检查结果呈阳性的概率为90%
- 若没有得这种癌症，检查结果呈阴性的概率为90%
- 该癌症的发病率是1%

$$\begin{bmatrix} TP \\ FP \\ TN \\ FN \end{bmatrix} = \begin{bmatrix} 0.009 \\ 0.099 \\ 0.891 \\ 0.001 \end{bmatrix}$$



总计1000人

$$\frac{TP}{TP + FP} = 8.33\%$$

小结

- **有很多性能度量指标可以用来评价二元分类效果**
  - 查准率P，查全率R， F1度量，等等
- **有很多性能度量指标可以用来评价（带有可变参数的）二元分类模型**
  - P-R曲线，平衡点，ROC曲线，AUC，等等
- **至少要4个数字指标，或者3个百分比指标，才能复现“实际/诊断-阳性/阴性”组合图，才能全面评价二元分类的效果**

扩展：多元分类评价指标

- **提问：多元分类怎么评价?**
  - 基本思路一：二元结果（分类正确 vs 分类不正确）
  - 基本思路二：多个二元分类（属于x类 vs 不属于x类）

多元分类评价指标

- 提问：多元分类怎么评价?

3.3.1. The `scoring` parameter: defining model evaluation rules

Model selection and evaluation using tools, such as `model_selection.GridSearchCV` and `model_selection.cross_val_score`, take a `scoring` parameter that controls what metric they apply to the estimators evaluated.

3.3.1.1. Common cases: predefined values

For the most common use cases, you can designate a scorer object with the `scoring` parameter; the table below shows all possible values. All scorer objects follow the convention that **higher return values are better than lower return values**. Thus metrics which measure the distance between the model and the data, like `metrics.mean_squared_error`, are available as neg_mean_squared_error which return the negated value of the metric.

Scoring	Function	Comment
Classification		
'accuracy'	`metrics.accuracy_score`	
'balanced_accuracy'	`metrics.balanced_accuracy_score`	
'top_k_accuracy'	`metrics.top_k_accuracy_score`	
'average_precision'	`metrics.average_precision_score`	
'neg_brier_score'	`metrics.brier_score_loss`	
'f1'	`metrics.f1_score`	for binary targets
'f1_micro'	`metrics.f1_score`	micro-averaged
'f1_macro'	`metrics.f1_score`	macro-averaged
'f1_weighted'	`metrics.f1_score`	weighted average
'f1_samples'	`metrics.f1_score`	by multilabel sample
'neg_log_loss'	`metrics.log_loss`	requires `predict_proba` support
'precision' etc.	`metrics.precision_score`	suffixes apply as with 'f1'
'recall' etc.	`metrics.recall_score`	suffixes apply as with 'f1'
'jaccard' etc.	`metrics.jaccard_score`	suffixes apply as with 'f1'
'roc_auc'	`metrics.roc_auc_score`	
'roc_auc_ovr'	`metrics.roc_auc_score`	
'roc_auc_ovo'	`metrics.roc_auc_score`	
'roc_auc_ovr_weighted'	`metrics.roc_auc_score`	
'roc_auc_ovo_weighted'	`metrics.roc_auc_score`	

多元分类评价指标

- 提问：多元分类怎么评价?

average : *{'micro', 'macro', 'samples', 'weighted', 'binary'} or None, default='binary'*

This parameter is required for multiclass/multilabel targets. If `None`, the scores for each class are returned. Otherwise, this determines the type of averaging performed on the data:

`'binary'`:
Only report results for the class specified by `pos_label`. This is applicable only if targets (`y_{true,pred}`) are binary.



`'micro'`:
Calculate metrics globally by counting the total true positives, false negatives and false positives.



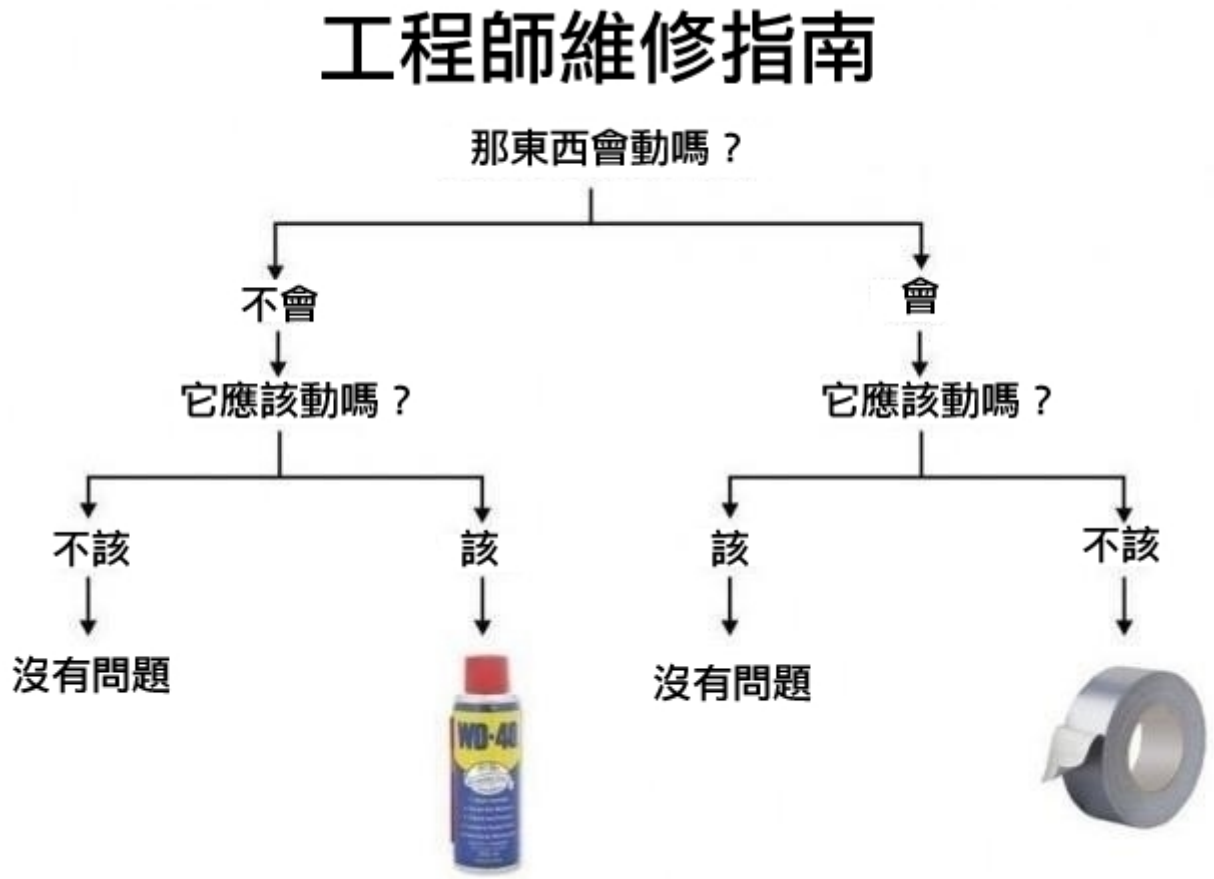
`'macro'`:
Calculate metrics for each label, and find their unweighted mean. This does not take label imbalance into account.

`'weighted'`:
Calculate metrics for each label, and find their average weighted by support (the number of true instances for each label). This alters 'macro' to account for label imbalance; it can result in an F-score that is not between precision and recall.

`'samples'`:
Calculate metrics for each instance, and find their average (only meaningful for multilabel classification where this differs from `accuracy_score`).

决策树模型
(decision tree models)

決策樹模型

- 

决策树模型

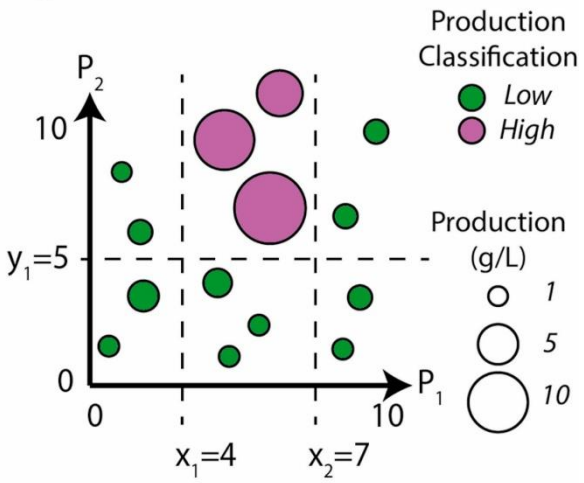
Lawson, Christopher E., Jose Manuel Martí, Tijana Radivojevic, Sai Vamshi R. Jonnalagadda, Reinhard Gentz, Nathan J. Hillson, Sean Peisert et al. "Machine learning for metabolic engineering: A review." *Metabolic Engineering* 63 (2021): 34-60.

A

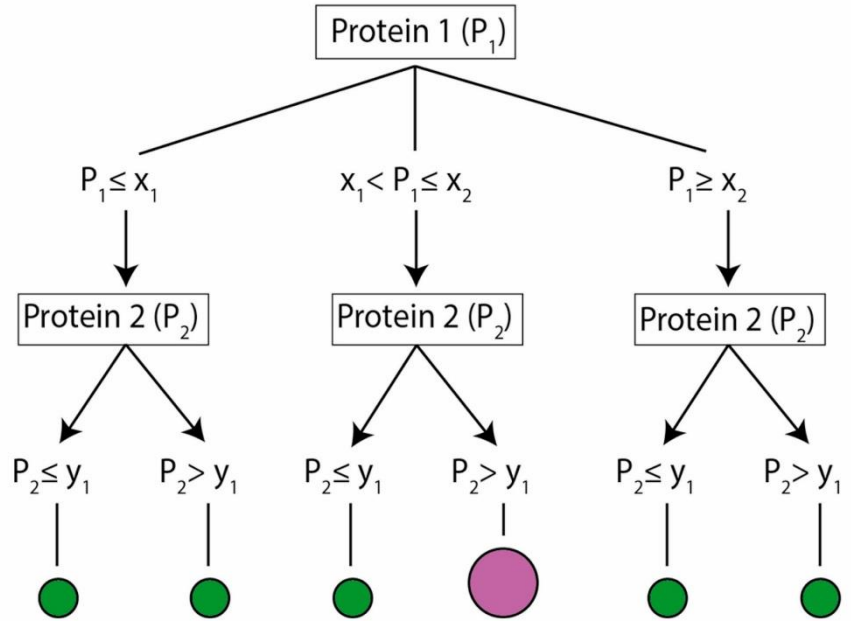
	Input features		Response
Strain ID	Protein 1 (units/cell)	Protein 2 (units/cell)	Production (Low/High)
s1	2	8	Low
s2	3	6	Low
s3	9	6	Low
s4	5	10	High
s5	10	10	Low
s6	5	1	Low
s7	6	2	Low
s8	9	1	Low
s9	6	7	High
s10	1	1	Low
s11	5	4	Low
s12	2	4	Low
s13	6	12	High
s14	9	3	Low
s15	4	7	?

Instances

B



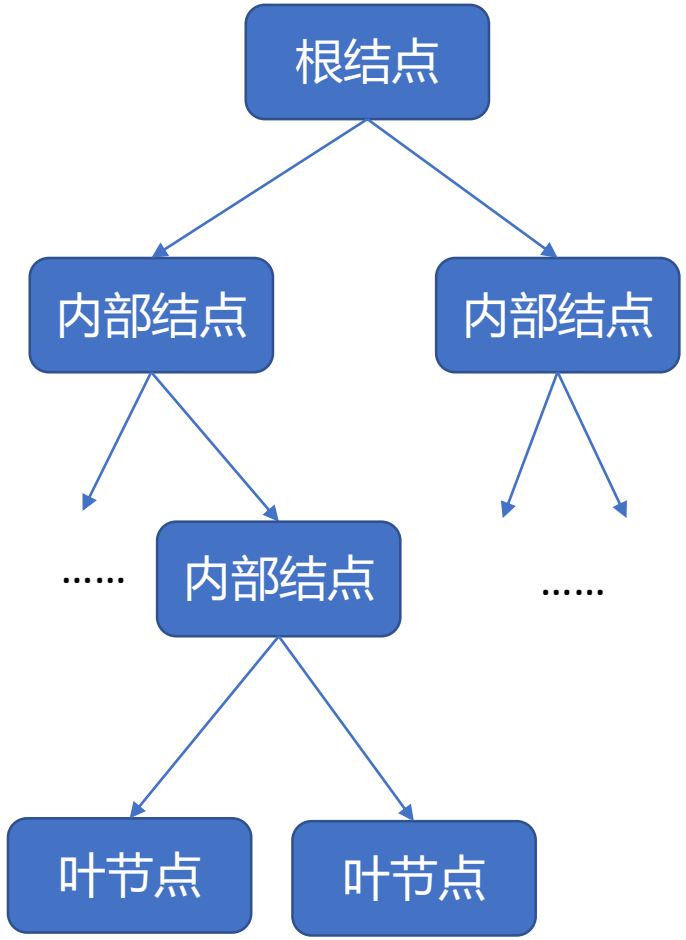
C



决策树模型

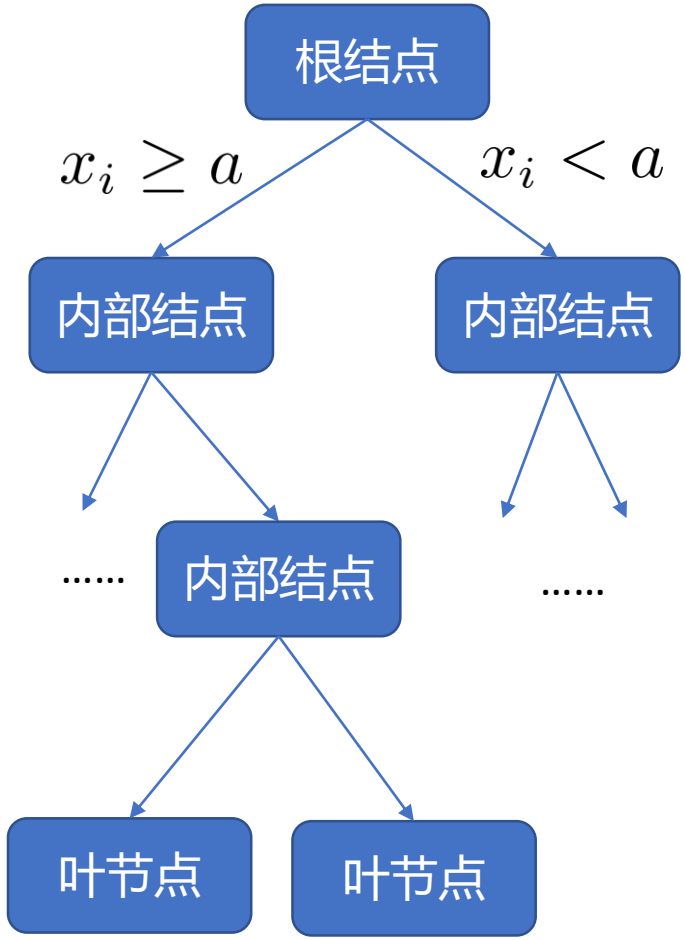
- **决策树模型常用于有监督学习中的分类问题**

- **一般的，一颗决策树包含**
  - 一个根结点（root node）
  - 若干个内部结点（node）
  - 根结点和每个内部结点可以有若干个分支（branch）
  - 每个分支的终端有一个叶结点（leaf）



决策树模型

- **每个结点对应一个特征测试**
- **样本根据特征测试结果，被划分到子结点中**
  - 每个结点，可以看成所有样本数据的一个子集
- **决策树学习（训练）的目的，是为了产生一颗泛化能力强，即处理未见示例能力强的树**



划分选择

- **训练决策树的关键就是在一个结点选择如何划分**
  - 试想我们从根结点开始，要逐一确定如何产生子结点

- **一般而言，我们希望决策树的分支结点所包含的样本尽可能属于同一类别，即结点的“纯度”越来越高**

几种常见的划分选择依据

- **信息增益（ID3决策树算法使用）**
  - 信息熵（entropy）是度量样本集合纯度的指标；我们希望划分形成子集后的信息熵更小（信息熵更小代表纯度更高）
- **增益率（C4.5决策树算法使用）**
  - 信息增益对可取值数目多的特征有偏好；增益率对信息增益进行改进，减少了这种不利影响
- **基尼指数（CART决策树算法使用）**
  - 基尼值：随机抽取两个样本，其类别标记不一致的概率；基尼指数使用基尼值描述划分的优劣

剪枝（pruning）

- **决策树也会出现过拟合问题；剪枝是决策树学习算法对付过拟合的主要手段**

- **预剪枝（pre-pruning）：在决策树的生成过程中，在每个结点划分前进行估计；若当前结点的划分不能带来泛化性能提升，则停止划分，并将当前结点标记为叶结点**
  - 如何知道能否带来泛化性能提升：可以使用验证数据集

剪枝

- 后剪枝（post-pruning）：生成了完整决策树之后，自底向上地对内部结点进行考察

- 若将该结点对应的子树替换为叶结点能提升泛化性能，则替换之

缺失值处理

- **目前为止我们的讨论都是针对高质量数据：每个样本的特征都是完整的**
- **然而，现实任务中常常会遇到不完整的样本**
  - 如由于诊测成本、隐私等因素，患者的医疗数据在某个检测项目的结果未知
  - 在特征数量多时，可能有大量样本出现缺失值
- **我们需要考虑利用有部分特征缺失的样本来训练决策树**

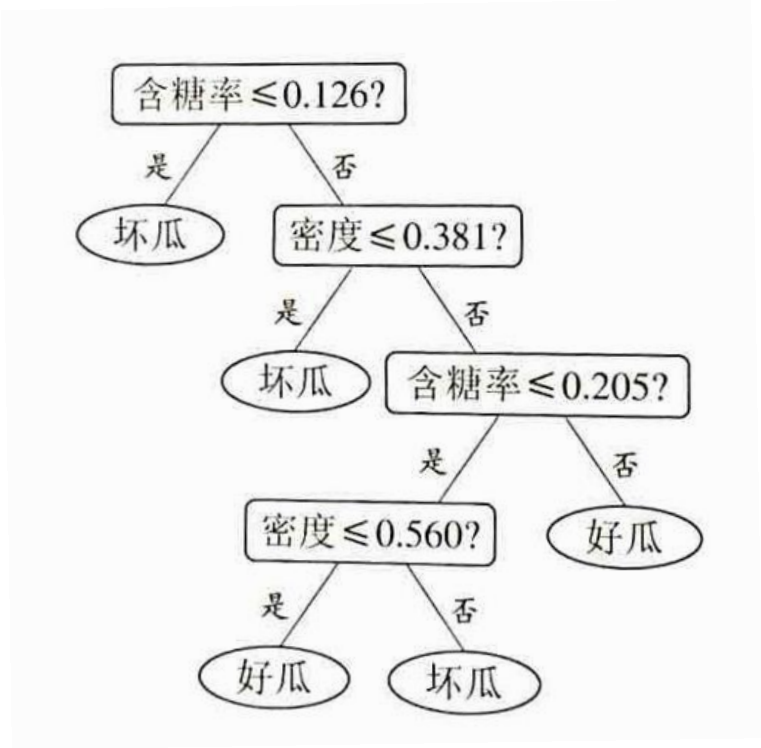
缺失值处理

- 在决策树学习中，若样本在划分属性上的取值未知，可以考虑将其“同时划入所有子结点”并更新相应权重

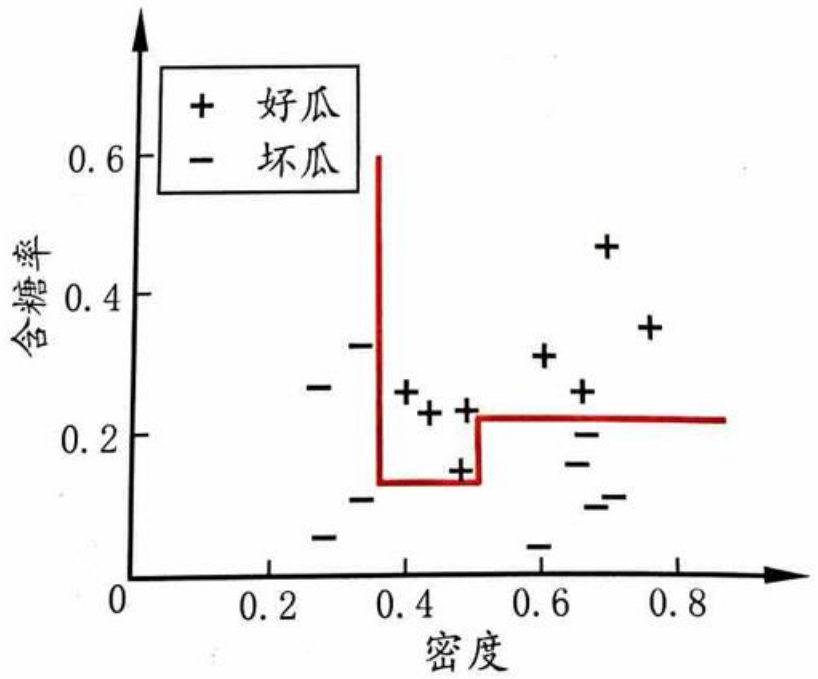
- 提问：在决策树推理中，如果有缺失值，我们可以怎么办？

单变量划分与多变量划分决策树

- **单变量决策：每个分支由一个变量决定**



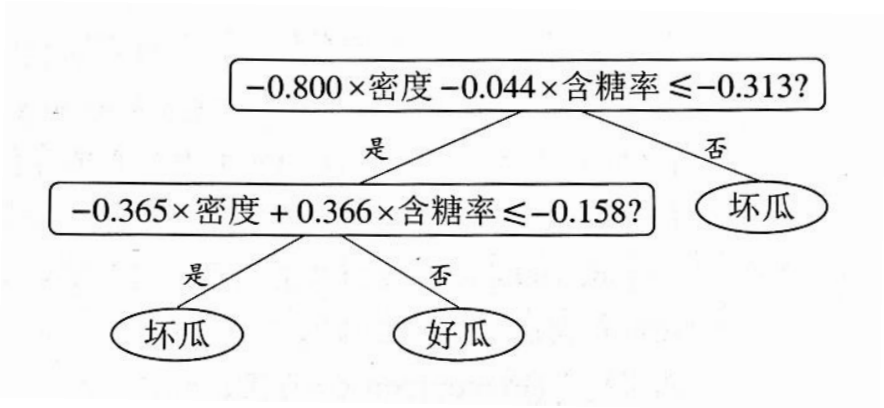
通过一系列单变量，判断“西瓜好不好”



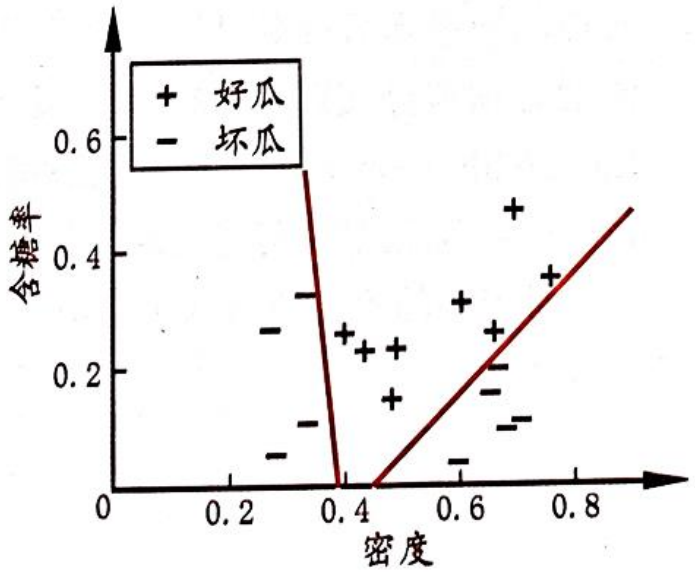
单变量决策树对应的分类边界

单变量划分与多变量划分决策树

- **多变量决策：每个分支由多个个变量决定**



通过一系列多变量划分，判断“西瓜好不好”



多变量决策树对应的分类边界

决策树：scikit-learn 示例

运行的代码

```
from sklearn import tree

X = [[0, 0], [1, 1]]
Y = [0, 1]

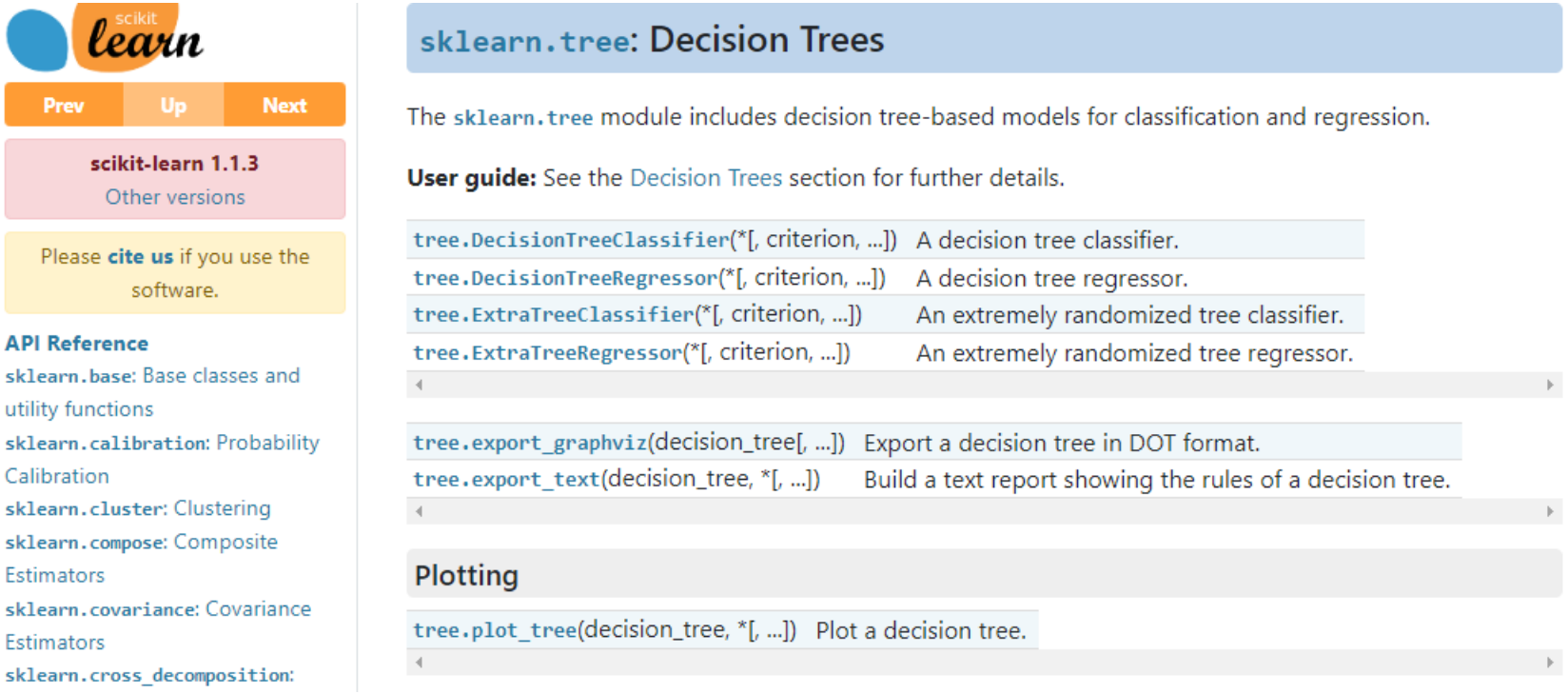
clf = tree.DecisionTreeClassifier()
clf = clf.fit(X, Y)

clf.predict([[2., 2.]])
```

返回值

```
array([1])
```

决策树：scikit-learn 示例



scikit learn

Prev | Up | Next

scikit-learn 1.1.3
Other versions

Please **cite us** if you use the software.

API Reference

`sklearn.base`: Base classes and utility functions
`sklearn.calibration`: Probability Calibration
`sklearn.cluster`: Clustering
`sklearn.compose`: Composite Estimators
`sklearn.covariance`: Covariance Estimators
`sklearn.cross_decomposition`:

`sklearn.tree`: Decision Trees

The `sklearn.tree` module includes decision tree-based models for classification and regression.

User guide: See the Decision Trees section for further details.

`tree.DecisionTreeClassifier`(*[, criterion, ...])	A decision tree classifier.
`tree.DecisionTreeRegressor`(*[, criterion, ...])	A decision tree regressor.
`tree.ExtraTreeClassifier`(*[, criterion, ...])	An extremely randomized tree classifier.
`tree.ExtraTreeRegressor`(*[, criterion, ...])	An extremely randomized tree regressor.

`tree.export_graphviz`(decision_tree[, ...])	Export a decision tree in DOT format.
`tree.export_text`(decision_tree, *[, ...])	Build a text report showing the rules of a decision tree.

Plotting

`tree.plot_tree`(decision_tree, *[, ...])	Plot a decision tree.

决策树：scikit-learn 示例

sklearn.tree.DecisionTreeClassifier

class sklearn.tree.**DecisionTreeClassifier**(*, *criterion='gini', splitter='best', max_depth=None, min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features=None, random_state=None, max_leaf_nodes=None, min_impurity_decrease=0.0, class_weight=None, ccp_alpha=0.0*) [source]

A decision tree classifier.

Read more in the User Guide.

Parameters::

criterion : *{"gini", "entropy", "log_loss"}, default="gini"*
The function to measure the quality of a split. Supported criteria are "gini" for the Gini impurity and "log_loss" and "entropy" both for the Shannon information gain, see Mathematical formulation.

splitter : *{"best", "random"}, default="best"*
The strategy used to choose the split at each node. Supported strategies are "best" to choose the best split and "random" to choose the best random split.

max_depth : *int, default=None*
The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than min_samples_split samples.

min_samples_split : *int or float, default=2*
The minimum number of samples required to split an internal node:

- If int, then consider `min_samples_split` as the minimum number.
- If float, then `min_samples_split` is a fraction and `ceil(min_samples_split * n_samples)` are the minimum number of samples for each split.

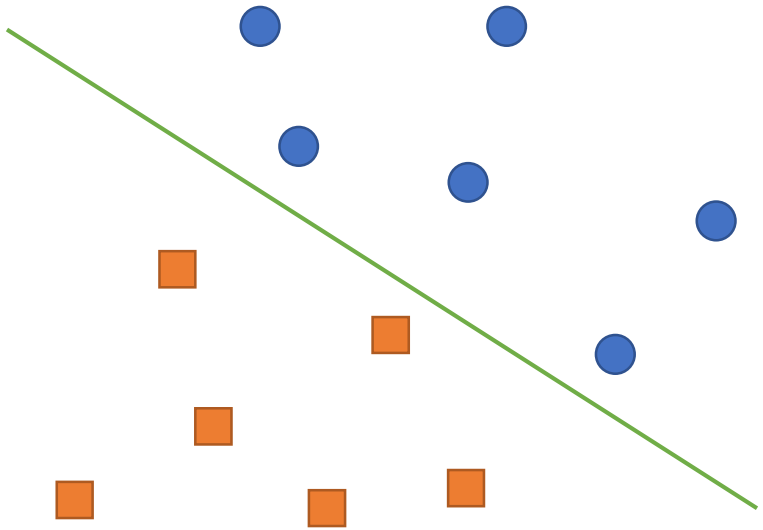
Changed in version 0.18: Added float values for fractions.

min_samples_leaf : *int or float, default=1*
The minimum number of samples required to be at a leaf node. A split point at any depth will only be considered if it leaves at least `min_samples_leaf` training samples in each of the left and right branches. This may have the effect of smoothing the model, especially in regression.

支持向量机模型
(Support Vector Machine, SVM)

二元分类

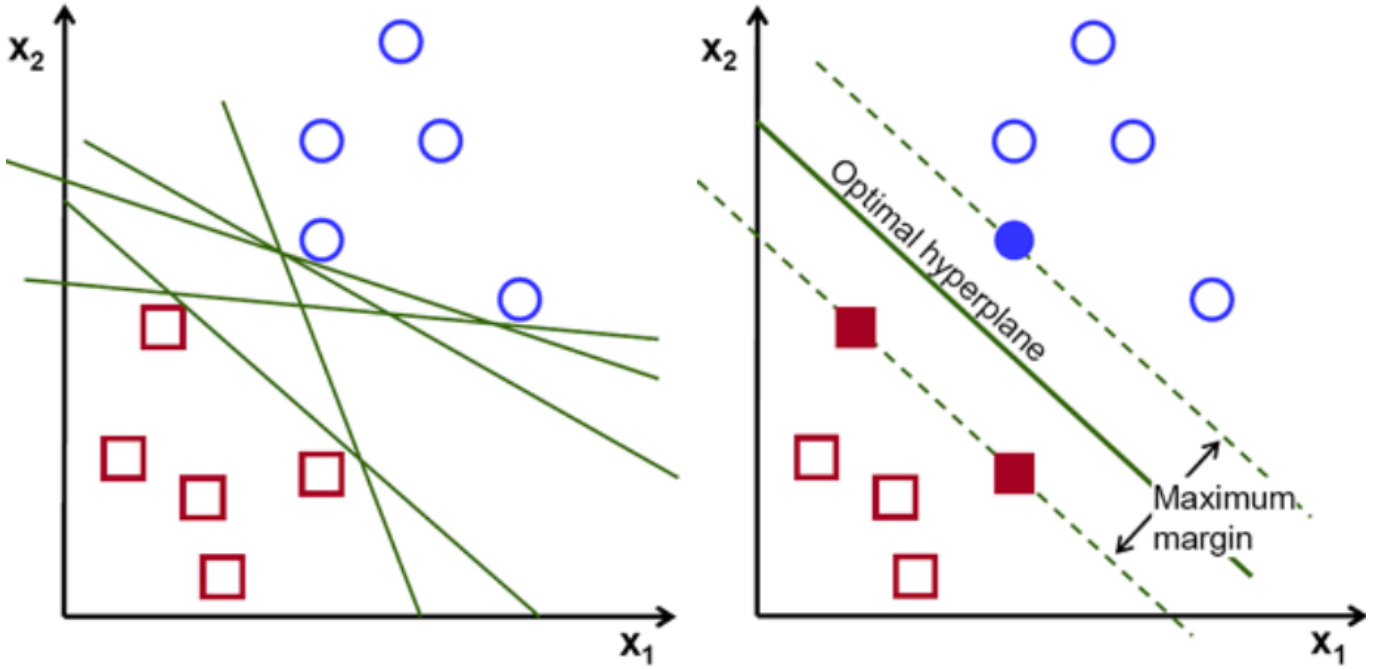
- 给定训练样本集$D = \{(\boldsymbol{x}_1, y_1), ..., (\boldsymbol{x}_m, y_m)\}$, $\boldsymbol{x}_i \in \mathbb{R}^d$, $y_i \in \{-1, +1\}$, 分类学习最基本的想法就是基于该训练集，在样本空间中找到一个划分超平面（hyperplane），将不同类样本分开



示例：$\boldsymbol{x}_i \in \mathbb{R}^2$，用于分割的超平面就是一条直线

二元分类：支持向量机

- **存在多个超平面将两类样本分开**
  - 哪一个超平面的最好的?



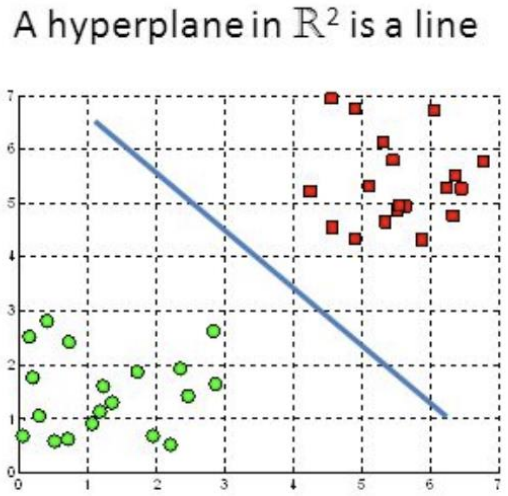
  - 找两类样本“正中间”的超平面划分，该超平面到两类样本的“最小距离”要尽量大

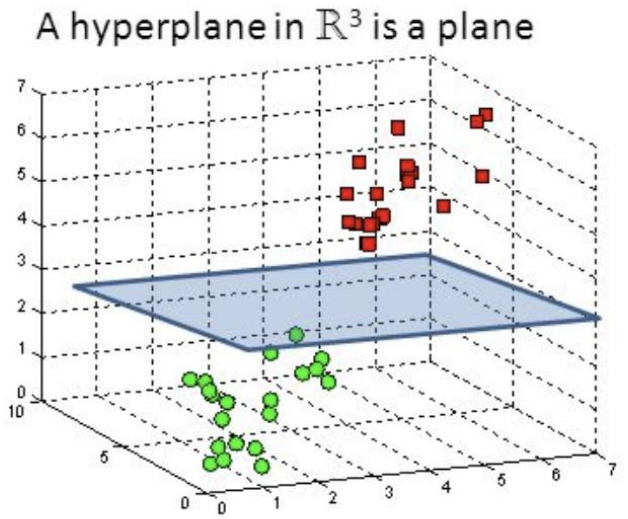
支持向量机

- 划分超平面可用如下线性方程来描述

$$\boldsymbol{w}^T\boldsymbol{x} + b = 0,$$

其中，$\boldsymbol{w} = \begin{bmatrix} w_1 & w_2 & \dots & w_d \end{bmatrix}^T$ 是法向量，决定了超平面的方向；b 为位移项，决定了超平面与原点之间的距离





支持向量机

- 超平面可被法向量 $\boldsymbol{w}$ 和位移 b 决定，因此我们将其记为 $(\boldsymbol{w}, b)$

- 样本空间中任意点 $\boldsymbol{x}$ 到超平面 $(\boldsymbol{w}, b)$ 的距离可写为

$$r = \frac{|\boldsymbol{w}^T \boldsymbol{x} + b|}{||\boldsymbol{w}||}$$

证明

- 取超平面L上的一点$M_1：x_1 = (x_1^{(1)}, x_1^{(2)}, x_1^{(3)}, ..., x_1^{(n)})$，有$x_1 \cdot w + b = 0$
- 超平面L的法向量为$w = (w^{(1)}, w^{(2)}, w^{(3)}, ..., w^{(n)})$

$$d = ||\overrightarrow{M_0 M_1}||cos(\overrightarrow{M_0 M_1}, w) = ||\overrightarrow{M_0 M_1}|| \frac{|\overrightarrow{M_0 M_1} \cdot w|}{||\overrightarrow{M_0 M_1}|| \, ||w||}$$

- $$= \frac{|(x_1 - x_0) \cdot w|}{||w||} = \frac{|x_1 \cdot w - x_0 \cdot w|}{||w||} = \frac{|-b - x_0 \cdot w|}{||w||}$$

$$= \frac{1}{||w||} |w \cdot x + b|$$

- 假设超平面 $(\boldsymbol{w}, b)$ 能将训练样本完全正确地分类，即对于 $(\boldsymbol{x}_i, y_i) \in D$，若 $y_i = +1$，则有 $\boldsymbol{w}^T \boldsymbol{x}_i + b_i > 0$；若 $y_i = -1$，则有 $\boldsymbol{w}^T \boldsymbol{x}_i + b_i < 0$

支持向量机

- 令

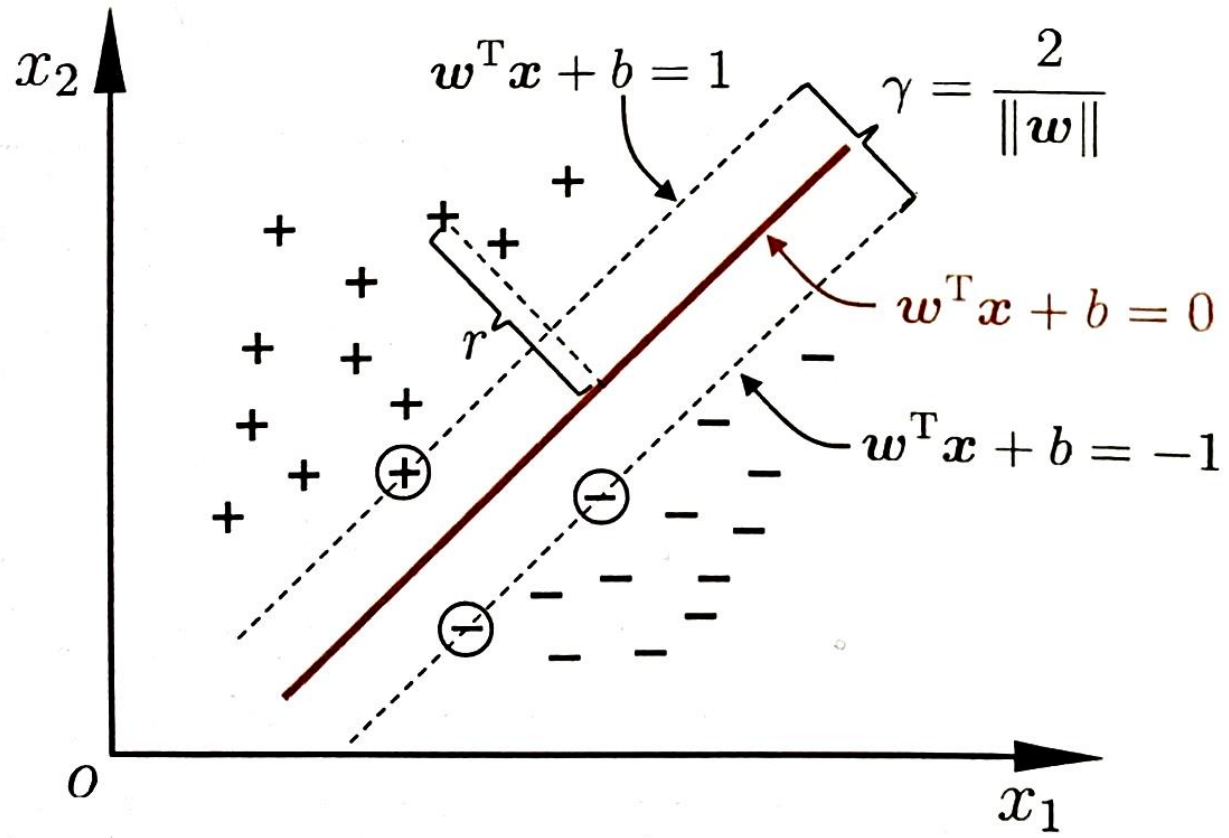
$$\begin{cases} \boldsymbol{w}^T \boldsymbol{x}_i + b \geq +1, y_i = +1 \\ \boldsymbol{w}^T \boldsymbol{x}_i + b \leq -1, y_i = -1 \end{cases}$$

距离超平面最近地几个训练样本，使上式的等号成立。它们被称为“支持向量”。两个异类支持向量到超平面的距离之和为

$$\gamma = \frac{2}{||\boldsymbol{w}||},$$

它被称之为“间隔”（margin）。

支持向量机

- 
x_2
$w^{\mathrm{T}}x + b = 1$
$\gamma = \frac{2}{\|w\|}$
r
$w^{\mathrm{T}}x + b = 0$
$w^{\mathrm{T}}x + b = -1$
O
x_1

支持向量机：模型训练

- 若要找到具有“最大间隔”（maximum margin）的划分超平面，也就是要找到使 γ 最大的参数 $\boldsymbol{w}$ 和 b，即

$$\begin{aligned} \max_{\boldsymbol{w},b} \quad & \frac{2}{||\boldsymbol{w}||} \\ \text{s.t.} \quad & \boldsymbol{w}^T\boldsymbol{x}_i + b \geq +1, \ \text{if } y_i = +1 \\ & \boldsymbol{w}^T\boldsymbol{x}_i + b \leq -1, \ \text{if } y_i = -1 \end{aligned}$$

以上优化问题亦可写成

$$\begin{aligned} \min_{\boldsymbol{w},b} \quad & ||\boldsymbol{w}||^2 \\ \text{s.t.} \quad & y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1, i = 1, 2, ..., m \end{aligned}$$

支持向量机：模型训练

- 给定训练集 D，求解 $(\boldsymbol{w}, b)$ 是凸优化问题，比较容易求解。人们也发明了针对求解SVM的更高效算法，如SMO（Sequential Minimal Optimization）等

支持向量机： scikit-learn 示例

运行的代码

```
from sklearn import svm

X = [[0, 0], [1, 1]]
Y = [0, 1]

clf = svm.SVC()
clf = clf.fit(X, Y)

clf.predict([[2., 2.]])
```

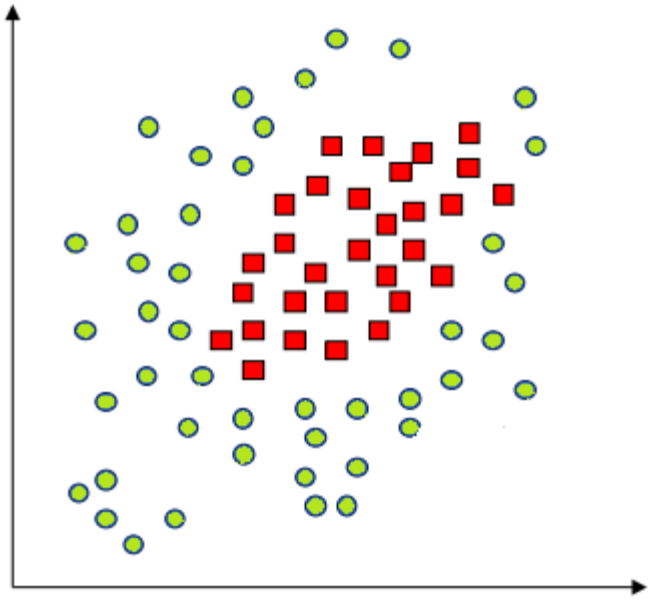
返回值

```
array([1])
```

核函数（kernel function）方法

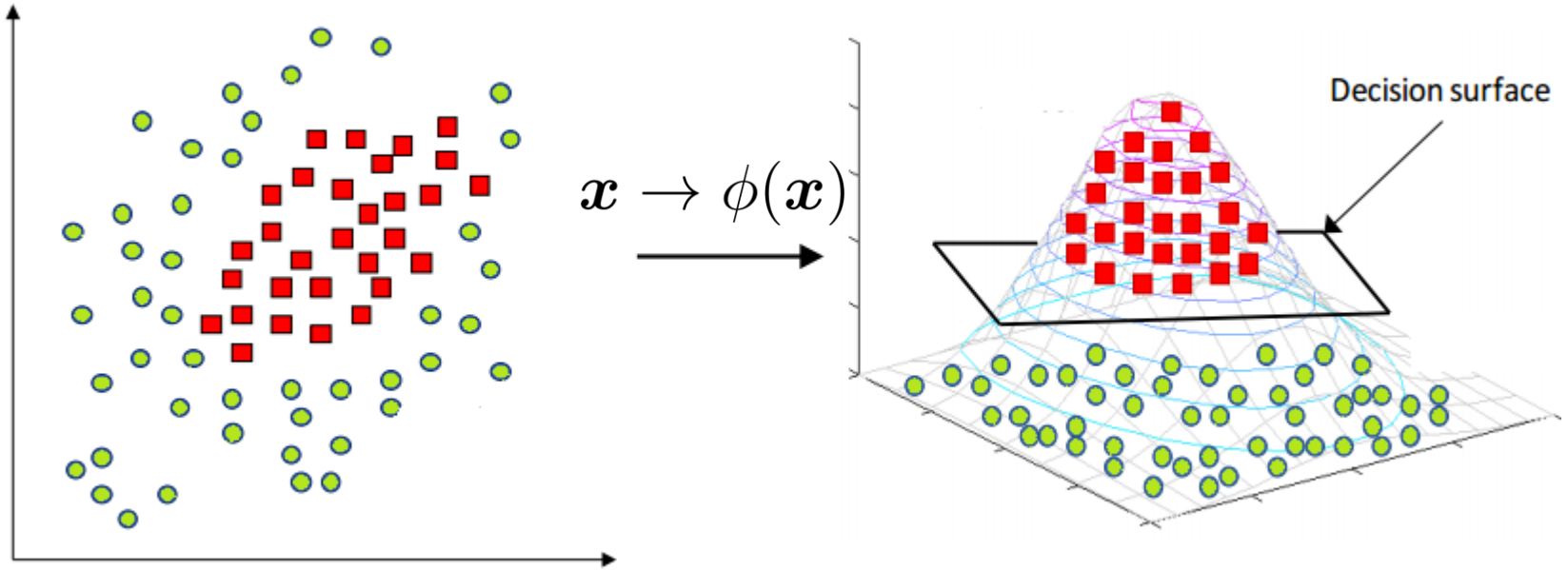
- 在针对支持向量机的讨论中，到目前为止，我们假设训练样本是“线性可分”的，即存在一个划分超平面，将训练样本分类正确

- 实际中，有可能不存在这样的能正确划分样本的超平面



核函数方法

- 我们可以考虑将原始空间中的每个样本 x 映射到一个更高维的特征空间，使得其在这个高维特征空间内线性可分



例如 $\boldsymbol{x} = [x_1, x_2]^T$, $\phi(\boldsymbol{x}) = [x_1, x_2, -(x_1^2 + x_2^2)]^T$.

核函数方法

- 令 $\phi(\boldsymbol{x})$ 表示将 $\boldsymbol{x}$ 映射后的特征向量，于是，在新特征空间中划分超平面所对应的模型可表示为

$$f(\boldsymbol{x}) = \boldsymbol{w}^T \phi(\boldsymbol{x}) + b,$$

其中 $\boldsymbol{w}$ 和 b 是超平面参数

- 求解以下优化问题可以得到最优参数 $\boldsymbol{w}$ 和 b

$$\begin{aligned} \min_{\boldsymbol{w},b} \quad & ||\boldsymbol{w}||^2 \\ \text{s.t.} \quad & y_i(\boldsymbol{w}^T \phi(\boldsymbol{x}_i) + b) \geq 1, i = 1, 2, ..., m \end{aligned}$$

核函数

- 显然，我们有个重要问题尚未解决：怎么选择 $\phi(\boldsymbol{x})$？

- 事实上，我们并不需要直接选择 $\phi(\boldsymbol{x})$. 在求解上页的优化问题时，我们只需要用到 $\phi(\boldsymbol{x}_i)^T\phi(\boldsymbol{x}_j)$，不需要直接用到 $\phi(\boldsymbol{x}_i)$ （详细计算过程此处略去）

- 我们定义核函数

$$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = \phi(\boldsymbol{x}_i)^T\phi(\boldsymbol{x}_j),$$

核函数也可以看作 $\phi(\boldsymbol{x}_i)$ 和 $\phi(\boldsymbol{x}_j)$ 的内积

核函数

- 给定 $\phi(\boldsymbol{x})$，我们可以计算核函数 $\kappa(\cdot,\cdot)$. 如果没有给定 $\phi(\boldsymbol{x})$，我们如何直接设计一个核函数呢？核函数需要满足什么特殊的要求吗？

核函数

- 令 $\mathcal{X}$ 为输入空间，$\kappa(\cdot,\cdot)$ 是定义在 $\mathcal{X}\times\mathcal{X}$ 上的对称函数，那么，κ 是核函数当且仅当对于任意数据 $D=\{\boldsymbol{x}_1,\boldsymbol{x}_2,...,\boldsymbol{x}_m\}$,核矩阵（kernel matrix）$\boldsymbol{K}$ 是半正定的(positive semi-definite)，

$$\boldsymbol{K}=\begin{bmatrix}\kappa(\boldsymbol{x}_1,\boldsymbol{x}_1) & \kappa(\boldsymbol{x}_1,\boldsymbol{x}_2) & \cdots & \kappa(\boldsymbol{x}_1,\boldsymbol{x}_m)\\ \kappa(\boldsymbol{x}_2,\boldsymbol{x}_1) & \kappa(\boldsymbol{x}_2,\boldsymbol{x}_2) & \cdots & \kappa(\boldsymbol{x}_2,\boldsymbol{x}_m)\\ \vdots & \vdots & \ddots & \vdots\\ \kappa(\boldsymbol{x}_m,\boldsymbol{x}_1) & \kappa(\boldsymbol{x}_m,\boldsymbol{x}_2) & \cdots & \kappa(\boldsymbol{x}_m,\boldsymbol{x}_m)\end{bmatrix}$$

常用核函数

- 实际应用中，我们通常在以下几种核函数中进行选择
  - 线性核：$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = \boldsymbol{x}_i^T \boldsymbol{x}_j$
  - 多项式核：$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = (\boldsymbol{x}_i^T \boldsymbol{x}_j)^d$，$d \geq 1$ 是多项式的次数
  - 高斯核：$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = e^{(-\frac{||\boldsymbol{x}_i - \boldsymbol{x}_j||^2}{2\sigma^2})}$，$\sigma > 0$ 是高斯核的带宽
  - 拉普拉斯核：$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = e^{(-\frac{||\boldsymbol{x}_i - \boldsymbol{x}_j||}{\sigma})}$，$\sigma > 0$
  - Sigmoid核：$\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = \tanh(\beta \boldsymbol{x}_i^T \boldsymbol{x}_j + \theta)$, $\beta > 0$, $\theta < 0$, $\tanh$ 是双曲正切函数

核函数

- 我们希望样本在特征空间内线性可分
- 核函数隐式地限定了这个特征空间

- 通常，我们难以提前知道那个核函数是合适的；我们可以尝试上页中的多个常用核函数，从中选择效果最好的

核函数： scikit-learn 示例

class `sklearn.svm.`**SVC**(*, *C=1.0*, *kernel='rbf'*, *degree=3*, *gamma='scale'*, *coef0=0.0*, *shrinking=True*, *probability=False*, *tol=0.001*, *cache_size=200*, *class_weight=None*, *verbose=False*, *max_iter=-1*, *decision_function_shape='ovr'*, *break_ties=False*, *random_state=None*) [source]

C-Support Vector Classification.

The implementation is based on libsvm. The fit time scales at least quadratically with the number of samples and may be impractical beyond tens of thousands of samples. For large datasets consider using `LinearSVC` or `SGDClassifier` instead, possibly after a `Nystroem` transformer.

The multiclass support is handled according to a one-vs-one scheme.

For details on the precise mathematical formulation of the provided kernel functions and how `gamma`, `coef0` and `degree` affect each other, see the corresponding section in the narrative documentation: Kernel functions.

Read more in the User Guide.

Parameters::

C : *float, default=1.0*
Regularization parameter. The strength of the regularization is inversely proportional to C. Must be strictly positive. The penalty is a squared l2 penalty.

kernel : *{'linear', 'poly', 'rbf', 'sigmoid', 'precomputed'} or callable, default='rbf'*
Specifies the kernel type to be used in the algorithm. If none is given, 'rbf' will be used. If a callable is given it is used to pre-compute the kernel matrix from data matrices; that matrix should be an array of shape `(n_samples, n_samples)`.

degree : *int, default=3*
Degree of the polynomial kernel function ('poly'). Ignored by all other kernels.

gamma : *{'scale', 'auto'} or float, default='scale'*
Kernel coefficient for 'rbf', 'poly' and 'sigmoid'.

- if `gamma='scale'` (default) is passed then it uses 1 / (n_features * X.var()) as value of gamma,
- if 'auto', uses 1 / n_features.

Changed in version 0.22: The default value of `gamma` changed from 'auto' to 'scale'.

coef0 : *float, default=0.0*
Independent term in kernel function. It is only significant in 'poly' and 'sigmoid'.

核函数：

1.4. Support Vector Machines
1.4.1. Classification
1.4.2. Regression
1.4.3. Density estimation, novelty detection
1.4.4. Complexity
1.4.5. Tips on Practical Use
1.4.6. Kernel functions
1.4.7. Mathematical formulation
1.4.8. Implementation details

1.4.6. Kernel functions

The *kernel function* can be any of the following:

- linear: $\langle x, x' \rangle$.
- polynomial: $(\gamma \langle x, x' \rangle + r)^d$, where d is specified by parameter `degree`, r by `coef0`.
- rbf: $\exp(-\gamma \|x - x'\|^2)$, where γ is specified by parameter `gamma`, must be greater than 0.
- sigmoid $\tanh(\gamma \langle x, x' \rangle + r)$, where r is specified by `coef0`.

Different kernels are specified by the `kernel` parameter:

```
>>> linear_svc = svm.SVC(kernel='linear')
>>> linear_svc.kernel
'linear'
>>> rbf_svc = svm.SVC(kernel='rbf')
>>> rbf_svc.kernel
'rbf'
```

See also Kernel Approximation for a solution to use RBF kernels that is much faster and more scalable.

1.4.6.1. Parameters of the RBF Kernel

When training an SVM with the *Radial Basis Function* (RBF) kernel, two parameters must be considered: `C` and `gamma`. The parameter `C`, common to all SVM kernels, trades off misclassification of training examples against simplicity of the decision surface. A low `C` makes the decision surface smooth, while a high `C` aims at classifying all training examples correctly. `gamma` defines how much influence a single training example has. The larger `gamma` is, the closer other examples must be to be affected.

Proper choice of `C` and `gamma` is critical to the SVM's performance. One is advised to use `GridSearchCV` with `C` and `gamma` spaced exponentially far apart to choose good values.

Examples:

- RBF SVM parameters
- Non-linear SVM

1.4.6.2. Custom Kernels

You can define your own kernels by either giving the kernel as a python function or by precomputing the Gram matrix.

核函数： scikit-learn 示例

运行的代码

```
from sklearn import svm

linear_svc = svm.SVC(kernel='linear’)

rbf_svc = svm.SVC(kernel='rbf')
```

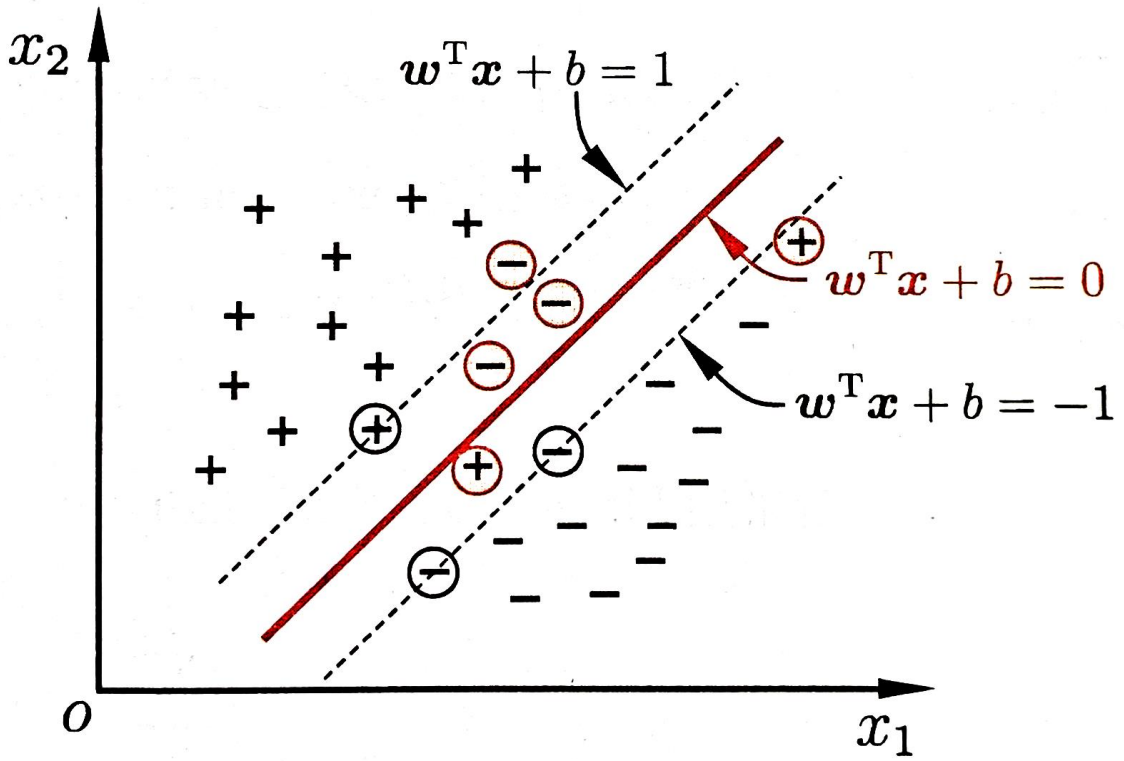
软间隔

- 我们一直假定样本在样本空间（无需核函数）或者特征空间（需核函数）中是线性可分的，即存在一个超平面，将两类样本完全分开

- 实际中，由于种种原因，很可能不存在这样的超平面

- 我们应该允许支持向量机在一部分样本上出错，为此，要引入“软间隔”（soft margin）概念

软间隔



从硬间隔到软间隔

- 在“硬间隔”中，我们把“分类正确”当作约束写进了支持向量机参数优化问题中：

$$\begin{aligned} \min_{\boldsymbol{w},b} \quad & ||\boldsymbol{w}||^2 \\ \text{s.t.} \quad & y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1, i = 1, 2, ..., m \end{aligned}$$

- 在“软间隔”中，我们要允许部分样本不满足 $y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1$；同时，也希望不满足该条件的样本尽量少

从硬间隔到软间隔

- 在“软间隔”情形下，优化目标可写为

$$\min_{\boldsymbol{w},b} \quad ||\boldsymbol{w}||^2 + C\sum_{i=1}^{m} l_{0/1}[y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) - 1],$$

其中 $C > 0$ 是权重，$l_{0/1}$ 是“0/1损失函数”

$$l_{0/1} = \begin{cases} 1, & \text{if } z < 0 \\ 0, & \text{otherwise.} \end{cases}$$

1. 满足约束条件的样本：损失为0
2. 不满足约束条件的样本：损失不为0

- $\mathrm{z} = \mathrm{y_i}\left(\vec{\mathrm{w}}^{\mathrm{T}}\vec{\mathrm{x}} + \mathrm{b}\right) - 1$
- 满足约束条件的样本:$\mathrm{z} \geq 0$，则损失=0
- 不满足约束条件的样本:$\mathrm{z} < 0$，则损失=1

从硬间隔到软间隔

$$l_{0/1} = \begin{cases} 1, & \text{if } z < 0 \\ 0, & \text{otherwise.} \end{cases}$$

- 然而，$l_{0/1}$ 非凸、不连续，数学性质不太好，使得该优化问题求解不便。于是我们用一些不同的“替代损失函数”来代替 $l_{0/1}$。替代损失函数通常是凸的、连续的。常用的替代损失函数如下
  - hinge 损失：$l_{hinge}(z) = \max(0, 1 - z)$
  - 指数损失（expotential loss）：$l_{exp}(z) = e^{-z}$
  - 对率损失（logistic loss）：$l_{log}(z) = \log(1 + e^{-z})$

scikit-learn 中的软间隔

- **scikit-learn 中的 SVM 模型默认都是使用的软间隔**

sklearn.svm.SVC

class **sklearn.svm.SVC**(*, *C=1.0*, *kernel='rbf'*, *degree=3*, *gamma='scale'*, *coef0=0.0*, *shrinking=True*, *probability=False*, *tol=0.001*, *cache_size=200*, *class_weight=None*, *verbose=False*, *max_iter=-1*, *decision_function_shape='ovr'*, *break_ties=False*, *random_state=None*) [source]

C-Support Vector Classification.

The implementation is based on libsvm. The fit time scales at least quadratically with the number of samples and may be impractical beyond tens of thousands of samples. For large datasets consider using `LinearSVC` or `SGDClassifier` instead, possibly after a `Nystroem` transformer.

The multiclass support is handled according to a one-vs-one scheme.

For details on the precise mathematical formulation of the provided kernel functions and how `gamma`, `coef0` and `degree` affect each other, see the corresponding section in the narrative documentation: Kernel functions.

Read more in the User Guide.

Parameters::

C : *float, default=1.0*
Regularization parameter. The strength of the regularization is inversely proportional to C. Must be strictly positive. The penalty is a squared l2 penalty.

scikit-learn 中的软间隔

- **scikit-learn 中的 SVM 模型默认都是使用的软间隔**

1.4.5. Tips on Practical Use

- **Avoiding data copy**: For `SVC`, `SVR`, `NuSVC` and `NuSVR`, if the data passed to certain methods is not C-ordered contiguous and double precision, it will be copied before calling the underlying C implementation. You can check whether a given numpy array is C-contiguous by inspecting its `flags` attribute.

  For `LinearSVC` (and `LogisticRegression`) any input passed as a numpy array will be copied and converted to the liblinear internal sparse data representation (double precision floats and int32 indices of non-zero components). If you want to fit a large-scale linear classifier without copying a dense numpy C-contiguous double precision array as input, we suggest to use the `SGDClassifier` class instead. The objective function can be configured to be almost the same as the `LinearSVC` model.

- **Kernel cache size**: For `SVC`, `SVR`, `NuSVC` and `NuSVR`, the size of the kernel cache has a strong impact on run times for larger problems. If you have enough RAM available, it is recommended to set `cache_size` to a higher value than the default of 200(MB), such as 500(MB) or 1000(MB).

- **Setting C**: `C` is `1` by default and it's a reasonable default choice. If you have a lot of noisy observations you should decrease it: decreasing C corresponds to more regularization.

  `LinearSVC` and `LinearSVR` are less sensitive to `C` when it becomes large, and prediction results stop improving after a certain threshold. Meanwhile, larger `C` values will take more time to train, sometimes up to 10 times longer, as shown in [[11]].

支持向量回归（support vector regression）

- 除了处理分类问题，支持向量机的方法也可用于回归模型

